

HiDiTingV100 DFX 软件

开发指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档主要介绍 HiDiTingV100 DFX 功能及使用指南，包括 DIAG 维测功能、死机诊断、Last Dump 解析等，方便用户进行业务维测及死机分析。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 软件工程师
- 技术支持工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 DIAG 维测功能.....1

1.1 概述..... 1

1.2 功能描述..... 1

1.3 日志打印功能 2

1.3.1 场景说明..... 2

1.3.2 工作流程..... 2

1.3.3 代码示例..... 4

1.4 日志离线保存功能 4

1.4.1 场景说明..... 4

1.4.2 工作流程..... 5

1.4.3 代码示例..... 6

1.5 命令注册功能 7

1.5.1 场景说明..... 7

1.5.2 工作流程..... 7

1.5.3 Database 修改 9

1.6 系统维测接口 10

1.6.1 内存使用统计 10

1.6.2 任务信息统计 11

2 死机诊断.....13

2.1 简介..... 13

2.2 死机信息获取 14

2.2.1 串口工具界面输出 15

2.2.2 Flash 中的死机文件获取	16
2.2.3 通过 DebugKits 工具获取	17
2.2.3.1 获取 last word 信息	18
2.2.3.2 获取 last dump 信息	19
2.2.4 连接 J-Link 调试器导出	20
2.3 死机信息查看	21
2.3.1 串口死机信息查看	21
2.3.1.1 异常信息汇总	22
2.3.1.2 CPU 寄存器信息	22
2.3.1.3 函数调用栈信息	24
2.3.2 Flash 中的死机文件信息查看	24
2.3.3 通过 DebugKits 获取的死机信息查看	27
2.3.3.1 last word 信息查看	27
2.3.3.2 last dump 信息查看	30
2.3.4 连接 J-Link 调试器查看信息	31
2.4 常见死机问题定位	34
2.4.1 看门狗重启问题定位	34
2.4.2 CPU 异常触发重启问题定位	34
2.4.2.1 Store 或 Load 异常	35
2.4.2.2 取指异常	35
3 Last Dump 解析	37
3.1 概述	37
3.2 功能描述	37
3.2.1 场景说明	37
3.2.2 工作流程	38
3.2.3 异常处理	41
4 内存 DFX 功能	44
4.1 概述	44
4.2 功能描述	44
4.2.1 查询指定任务内存占用情况	44
4.2.2 查询当前所有任务内存占用情况	46

4.2.3 查询当前所有空闲内存信息..... 48

4.2.4 查询当前内存统计信息..... 50

4.2.5 查询内存的所有信息..... 51

4.2.6 查询任务信息 52

4.2.7 查询任务水线 55

A DebugKits 使用说明57

插图目录

图 1-1 内存统计使用示例	10
图 1-2 任务信息统计示例	11
图 2-1 串口连接	15
图 2-2 串口工具中捕获的死机信息	16
图 2-3 死机文件导出方法	17
图 2-4 DebugKits 日志打印	18
图 2-5 A 核 last word 信息	18
图 2-6 BT 核 last word 信息	19
图 2-7 last dump 生成文件	20
图 2-8 J-Link 连接 MCPU 示意图	21
图 2-9 死机相关 CPU 寄存器说明	23
图 2-10 Linux 环境运行死机脚本工具	25
图 2-11 Windows 环境运行死机脚本工具	26
图 2-12 last dump 中的死机信息	31
图 2-13 查询 CPU 寄存器值	33
图 2-14 变量地址	33
图 2-15 变量值	33
图 3-1 DebugKits 工具 Dump 解析界面	39
图 3-2 Last Dump 解析结果文件	40
图 3-3 Last Dump 解析结果片段	41

图 3-4 config.ini 中配置信息	42
图 3-5 g_mem_dump_info 数组.....	42
图 3-6 DebugKits 工具调用解析脚本的命令	43
图 3-7 Last Dump 解析在命令行窗口执行结果.....	43
图 4-1 串口命令	45
图 4-2 命令执行结果	46
图 4-3 串口命令	47
图 4-4 命令执行结果	48
图 4-5 串口命令	49
图 4-6 命令执行结果	49
图 4-7 串口命令	50
图 4-8 命令执行结果	51
图 4-9 串口命令	52
图 4-10 串口命令	53
图 4-11 命令执行结果	54
图 4-12 串口命令	55
图 4-13 命令执行结果	56
图 A-1 选择芯片	57
图 A-2 连接芯片	58
图 A-3 配置 database.....	59
图 A-4 打开消息界面.....	59
图 A-5 查看日志	60
图 A-6 打开命令行界面	60
图 A-7 命令输入	61

表格目录

表 1-1 日志打印接口 2

表 2-1 死机信息类型说明 13

表 2-2 异常信息汇总 22

表 2-3 死机相关 CPU 寄存器说明 22

表 2-4 mcause&ccause 异常描述表 23

表 2-5 last word 内容 27

表 2-6 A 核 last word 中 reg_value 成员含义 28

表 2-7 BT 核 last word 中 reg_value 成员含义 29

表 2-8 J-Link 常用命令 32

表 2-9 看门狗死机关键信息 34

表 2-10 CPU 异常的 fault_type 含义 34

表 2-11 store 或 load 异常关键信息 35

1 DIAG 维测功能

- 1.1 概述
- 1.2 功能描述
- 1.3 日志打印功能
- 1.4 日志离线保存功能
- 1.5 命令注册功能
- 1.6 系统维测接口

1.1 概述

DIAG 维测功能是基于单板与 DebugKits 工具的交互，来支持单板诊断业务和其他维测相关业务开发。

1.2 功能描述

DIAG 维测功能模块提供以下功能：

- 日志打印。用户可以通过日志打印接口将调试信息打印到 DebugKits 的 Message 界面。
- 离线日志保存。在没有串口的场景下，DIAG 日志可以保存至 Flash 上，需要时导出至 PC 上进行解析。
- 命令注册。用户可以注册自定义命令，在 DebugKits 工具的命令行界面输入命令与单板侧进行交互，执行用户自定义的操作。

- 系统维测信息获取。DIAG 支持获取系统统计类信息（如内存使用、任务信息），帮助用户定位问题。

1.3 日志打印功能

1.3.1 场景说明

用户需要增加调试日志进行定位问题时，可以通过 DIAG 提供的日志打印接口，将调试信息打印到 DebugKits 的 Message 界面上。

1.3.2 工作流程

以用户在 “application/brandy/brandy_standard/main.c” 中增加调试日志为例，流程如下：

- 步骤 1 在 “application/brandy/brandy_standard/main.c” 中调用日志打印接口，输出调试信息。需要包含 “diag_log.h” 头文件。

表1-1 日志打印接口

函数	说明
uapi_diag_error_log	输出 ERROR 级别的调试日志（可变参数），最多带 10 个参数。
uapi_diag_warning_log	输出 WARNING 级别的调试日志（可变参数），最多带 10 个参数。
uapi_diag_info_log	输出 INFO 级别的调试日志（可变参数），最多带 10 个参数。
uapi_diag_debug_log	输出 DEBUG 级别的调试日志（可变参数），最多带 10 个参数。

- 步骤 2 用户在使用 DIAG 日志之前，需要先确认使用的模块。在 “application/brandy/brandy_standard/main.c” 文件中默认使用的是 LOG_PFMODULE 模块，模块 ID 的定义详见 “middleware/utis/dfx/log/include/log_module_id.h” 文件。

如果用户在自定义的目录，自己的代码中使用日志，需要在该目录的“CMakeLists.txt”文件中增加如下命令：

```
set(MODULE_NAME "app")  
set(AUTO_DEF_FILE_ID TRUE)
```

其中：“app”为模块名称，表示用户在这个组件中使用 app 模块（模块 ID 为 LOG_APPMODULE）的日志。

步骤 3 定义该文件的文件 ID，并添加到对应模块的文件 ID 列表文件中。

如：

- PF 模块的 ID 列表文件：middleware/chips/brandy/dfx/include/log_def_pf.h
- APP 模块的 ID 列表文件：middleware/chips/brandy/dfx/include/log_def_app.h

其他模块依次类推。

文件 ID 的格式：调用日志接口的文件的文件名大写并加“_C”后缀，如：MAIN_C。

步骤 4 编译程序，编译过程中将生成“output/brandy/database_evb”目录。

步骤 5 参照“A DebugKits 使用说明”中步骤三，将生成的数据库（database_evb）更新至 DebugKits 的数据库中。打开 DebugKits 的 Message 界面查看日志信息（详情请参见“A DebugKits 使用说明”中的步骤四）。

步骤 6 如要更改默认的日志级别以及打开或关闭某些模块的日志，可通过修改“middleware/chips/brandy/dfx/zdiag_adapt_sdt.c”文件中的 diag_auto_log_report_enable 函数。如下图中的两个数组，分别是默认打开的日志级别和模块。用户可根据实际情况修改。

```
void diag_auto_log_report_enable(void)
{
    uint32_t level[] = {
        DIAG_LOG_LEVEL_DBG,
        DIAG_LOG_LEVEL_INFO,
        DIAG_LOG_LEVEL_WARN,
        DIAG_LOG_LEVEL_ERROR,
    };
    uint32_t mod_id[] = {
        LOG_BTMODULE,
        LOG_DSPMODULE,
        LOG_PFMODULE,
        LOG_MEDIAMODULE,
        LOG_APPMODULE,
        LOG_GUIMODULE,
        LOG_STATUS_MODULE,
        LOG_OTA_MODULE,
        LOG_OHOSMODULE,
        LOG_BTHMODULE,
    };
};
```

----结束

1.3.3 代码示例

```
#include "diag_log.h"
int main()
{
    uapi_diag_info_log(0, "test out info log. value = %d", 1);
    uapi_diag_error_log(0, "test out error log. a = %d, b = %d\r\n", 3, 4);
    uapi_diag_warning_log(0, "test out warning log. a = %d, b = %d c = 0x%x\r\n", 3, 4, 5);
}
```

说明

请注意，DIAG 日志只能支持长度 32 位及以下的参数，如：“%d”、“%u”、“%x”、“%p”，无法支持长度大于 32 位的参数，如“%ld”、“%s”，无法支持浮点参数，如“%f”。

1.4 日志离线保存功能

1.4.1 场景说明

如“1.3 日志打印功能”中说明，DIAG 日志可以通过串口发送至 DebugKits 工具。在没有串口的场景下，可以打开日志离线存储功能，将 DIAG 日志以文件的形式保存至文件系统中，需要的时候将日志文件导出到 PC 上，通过 DebugKits 工具解析。

1.4.2 工作流程

步骤 1 打印日志的方法参见“[1.3 日志打印功能](#)”。

步骤 2 打开/关闭离线日志存储功能特性方法：

特性宏路径：middleware/chips/brandy/dfx/include/dfx_feature_config.h

示例如下：

```
/* 特性：支持日志离线存储到本地 */  
  
#ifndef CONFIG_DFX_SUPPORT_OFFLINE_LOG_FILE  
#define CONFIG_DFX_SUPPORT_OFFLINE_LOG_FILE          DFX_YES  
#endif
```

在此文件中将“CONFIG_DFX_SUPPORT_OFFLINE_LOG_FILE”的值置为“DFX_YES” 离线日志存储功能特性打开；置为“DFX_NO”，功能特性关闭。

步骤 3 打开/关闭日志离线存储功能的方法如下：

- 方法一：在代码中调用“uapi_zdiag_set_offline_log_enable”函数。
- 方法二：在 DebugKits 的命令行界面输入“offline_log_cfg”命令：

offline_log_cfg 1：打开日志离线保存功能。

offline_log_cfg 0：关闭日志离线保存功能。

```
>offline_log_cfg 1  
  
(0x7183)  
struct root =  
.str       = Offline log Enabled  
  
>offline_log_cfg 0  
  
(0x7183)  
struct root =  
.str       = Offline log Disabled
```

- 方法三：修改离线日志开关的 NV 项默认值。

在 NV 的预置值配置文件（如：

middleware/chips/brandy/nv/nv_config/cfg/acore/app.json）中将离线日志开关的 NV 项（offline_enable_flag）的值改为 1。

```
"offline_enable_flag":{  
    "key_id": "0x2002",
```

```
"key_status": "alive",  
"structure_type": "uint8_t",  
"attributions": 1,  
"value": 1  
},
```

打开日志离线保存功能后，DIAG 日志不再输出到串口，而是保存到文件中。

关闭日志离线保存功能后，DIAG 日志直接输出到串口。

说明

使用方法一和方法三，每次启动时，离线日志自动生效。方法二，只是临时生效，复位后会恢复。

步骤 4 从 `"/user/log/"` 目录导出所有日志文件。其中包含：

- 一个日志索引文件（log_file_diag.bin.idx）
- 若干个日志数据文件（如 log_file_diag.bin.0001、log_file_diag.bin.0002 等）

步骤 5 将日志文件拖入到 DebugKits 工具，可以查看日志内容（详情请参见《HiDiTingV100 DebugKits 工具 使用指南》）。

-----结束

说明

用于保存日志的空间有大小限制，目前总大小为 20MB，日志保存达到最大值后，会覆盖最早的日志继续保存。

1.4.3 代码示例

```
#include "diag_log.h"  
int main()  
{  
    uapi_zdiag_set_offline_log_enable(true);  
    uapi_diag_info_log(0, "test out value = %d", 1);  
    uapi_diag_error_log(0, "test out error a = %d, b = %d\\n", 3, 4);  
}
```

1.5 命令注册功能

1.5.1 场景说明

用户可以注册自定义命令，在 DebugKits 工具的命令界面输入命令与单板侧进行交互，执行用户自定义的操作。

1.5.2 工作流程

要想实现通过 DIAG 命令来控制单板的操作，需要 DIAG（单板侧）和 DebugKits（工具侧）约定命令和应答的 ID，以及命令和应答所发送的数据的数据结构。这些约定需要通过预先配置的 XML 文件来定义。

以增加一条 “get_user_info” 命令为例，流程如下：

步骤 1 在 “middleware/chips/brandy/dfx/include/soc_diag_cmd_id.h” 文件中定义命令 ID。

用户自定义命令范围 0xA0 ~ 0xEF，注意不要与已存在的命令 ID 重复。

代码示例如下：

```
#define DIAG_CMD_GET_USER_INFO          diag_cmd_id_comb(0xA0)
```

步骤 2 调用 “uapi_diag_register_cmd” 函数，注册该命令的回调函数。

其中第一个参数 “cmd_tbl” 表示注册的命令表格，“cmd_num” 表示命令表格中命令个数。可参考 “middleware/chips/brandy/dfx/dfx_system_init.c” 文件中 “register_default_diag_cmd” 函数的实现。

代码示例如下：

```
zdiag_cmd_reg_obj_t g_diag_user_cmd_tbl[] = {
    { DIAG_CMD_GET_USER_INFO, DIAG_CMD_GET_USER_INFO, diag_cmd_get_user_info },
};

static errcode_t register_diag_user_cmd(void)
{
    return uapi_zdiag_register_cmd(g_diag_user_cmd_tbl,
        sizeof(g_diag_user_cmd_tbl) / sizeof(g_diag_user_cmd_tbl[0]));
}
```

步骤 3 在步骤 2 中注册的回调函数 “diag_cmd_get_user_info” 中，实现收到该命令后的操作，如果需要给 DebugKits 工具应答，则调用 “uapi_diag_report_packet” 函数上报应答报文。

代码示例如下：

```
errcode_t diag_cmd_get_mem_info(uint16_t cmd_id, void * cmd_param, uint16_t cmd_param_size,
diag_option_t *option)
{
    errcode_t ret;
    user_info_t info;

    uapi_unused(cmd_param);
    uapi_unused(cmd_param_size);

    /* 获取用户的数据信息 */

    ret = dfx_mem_get_sys_user_info(&info);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }

    /* 将用户的数据信息发送给DebugKits */

    uapi_diag_report_packet(cmd_id, option, (uint8_t *)&info, (uint16_t)sizeof(user_info_t), true);
    return ERRCODE_SUCC;
}
```

步骤 4 在

“build/config/target_config/brandy/hdb_config/database_template/acore/system/hdbcfg/mss_cmd_db.xml” 中定义命令。代码示例如下：

```
<DebugKits>
  <GROUP NAME="FIX" DATA_STRUCT_FILE=".\\diag\\fix_struct_def.txt" MULTIMODE="Firefly"
  AUTO_STRUCT="YES" PLUGIN="0x111,0x110(1),0x252">
    <CMD ID="0xA005" NAME="get_user_info" DESCRIPTION="get_user_info" PLUGIN="0x100"
    TYPE="REQ_IND">
      <REQ STRUCTURE="tool_null_stru" TYPE="Auto" PARAM_VALUE="" />
      <IND STRUCTURE="user_info_t" TYPE="Auto" RESULT_CODE="" />
    </CMD>
  </GROUP>
</DebugKits>
```

步骤 5 编译程序，将编译生成的 “output/brandy/database_evb” 数据库更新至 DebugKits 工具数据库中（参照 “[A DebugKits 使用说明](#)” 中的[步骤三](#)），打开 DebugKits 命令行界面，即可输入 “get_user_info” 命令，实现获取用户数据信息的功能。

----结束

1.5.3 Database 修改

上述工作流程中的“步骤 4”修改的“mss_cmd_db.xml”文件，是 DIAG 与 DebugKits 约定数据结构的关键。下面对其详细说明：

- mss_cmd_db.xml 中分为两层：GROUP 和 CMD，每个 GROUP 可以包含多个 CMD 命令。
- GROUP 中的 DATA_STRUCTURE_FILE 字段指向的文件名，表示该 GROUP 下的命令涉及到的数据结构体都保存在这个文件中。
- CMD 中的 REQ STRUCTURE 字段指向请求命令携带数据对应的结构体，IND STRUCTURE 指向应答命令携带的数据对应的结构体。
- CMD 中 PLUGIN 字段表示该命令在 DebugKits 生效的界面：
 - 0x100：命令行页面
 - 0x259：system 页面
 - 0x110：message 页面
- GROUP 中的 AUTO_STRUCTURE 字段用来配置结构体是否自动生成，YES 表示构建过程中自动生成 database 的结构体，否则需要自己手动添加到结构体到 DATA_STRUCTURE_FILE 字段指向的文件中。例如：

/*mss_cmd_db.xml中添加一条get_user_info命令，CMD ID为0xA005*/

```
<CMD ID="0xA005" NAME="get_user_info" DESCRIPTION="get_user_info"
PLUGIN="0x100,0x102" TYPE="REQ_IND">
  <REQ STRUCTURE="tool_null_stru" TYPE="Auto" PARAM_VALUE="" />
  <IND STRUCTURE="mdm_user_info_t" TYPE="Auto" RESULT_CODE="" />
</CMD>
```

/*fix_struct_def.txt中添加结构体定义*/

```
typedef struct {
    uint32_t user_info1;
    uint32_t user_info2;
    uint32_t user_info3;
    uint32_t user_info4;
    uint32_t user_info5;
    uint32_t user_info6;
} user_info_t;
```

1.6 系统维测接口

系统维测功能提供了以下接口：

- 内存使用统计接口
- 任务信息统计接口

1.6.1 内存使用统计

概述

内存使用统计功能为用户提供内存使用的信息查询接口，可实时获取内存使用大小、峰值占用等信息。

功能描述

DIAG 提供内存使用统计信息查询接口 “diag_cmd_get_mem_info”，调用该接口可获取系统内存池总大小、已用空间、剩余空间、剩余空间节点个数、内存池已经使用的节点个数、内存池剩余空间中最大节点的大小以及内存池使用峰值等信息。以上信息以 “mdm_mem_info_t” 结构体的格式，发送到 DebugKits 中显示出来。如图 1-1 所示。

图1-1 内存统计使用示例

```
typedef struct {
    uint32_t total;          /* Total space of the memory pool (unit: byte).
                             CNcomment:内存池总大小（单位：byte）CNend */
    uint32_t used;           /* Used space of the memory pool (unit: byte).
                             CNcomment:内存池已经使用大小（单位：byte）CNend */
    uint32_t free;           /* Free space of the memory pool (unit: byte).
                             CNcomment:内存池剩余空间（单位：byte）CNend */
    uint32_t free_node_num;  /* Number of free nodes in the memory pool.
                             CNcomment:内存池剩余空间节点个数 CNend */
    uint32_t used_node_num;  /* Number of used nodes in the memory pool.
                             CNcomment:内存池已经使用的节点个数 CNend */
    uint32_t max_free_node_size; /* Maximum size of the node in the free space of the memory pool (unit: byte).
                             CNcomment:内存池剩余空间节点中最大节点的大小（单位：byte）CNend */
    uint32_t peak_size;      /* Peak memory usage of the memory pool.CNcomment:内存池使用峰值CNend */
} mdm_mem_info_t;
```

命令行代码示例

```
errcode_t diag_cmd_get_mem_info(uint16_t cmd_id, void * cmd_param, uint16_t cmd_param_size,
diag_option *option)
{
    errcode_t ret;
    mdm_mem_info_t info;
    uapi_unused(cmd_param);
```

```

uapi_unused(cmd_param_size);
ret = dfx_mem_get_sys_pool_info(&info);
if (ret != ERRCODE_SUCC) {
    return ret;
}
uapi_diag_report_packet(cmd_id, option, (uint8_t)&info, (uint16_t)sizeof(mdm_mem_info_t),
true);
return ERRCODE_SUCC;
}

```

1.6.2 任务信息统计

概述

任务信息统计功能可查询任务名、任务状态、当前 SP、优先级、栈峰值、栈大小等信息。

功能描述

提供任务统计功能查询接口 “diag_cmd_get_task_info”，调用该接口可实时获取当前系统任务 ID、任务状态、任务优先级、信号量、栈峰值、栈大小等信息。上述数据以 “task_info_t” 结构体的格式，发送到 DebugKits 中显示出来。如图 1-2 所示。

图1-2 任务信息统计示例

```

typedef struct {
    char name[EXT_TASK_NAME_LEN]; /* Task entrance function.CNcomment:入口函数CNend */
    bool valid;
    uint32_t id; /* Task ID.CNcomment:任务ID CNend */
    uint16_t status; /* Task status. Status detail see los_task_pri.h.CNcomment:任务状态.
    详细状态码请参考los_task_pri.h CNend */
    uint16_t priority; /* Task priority.CNcomment:任务优先级 CNend */
    void *task_sem; /* Semaphore pointer.CNcomment:信号量指针CNend */
    void *task_mutex; /* Mutex pointer.CNcomment:互斥锁指针CNend */
    uint32_t event_stru[3]; /* Event: 3 nums.CNcomment:3个事件CNend */
    uint32_t event_mask; /* Event mask.CNcomment:事件掩码CNend */
    uint32_t stack_size; /* Task stack size.CNcomment:栈大小CNend */
    uint32_t top_of_stack; /* Task stack top.CNcomment:栈顶CNend */
    uint32_t bottom_of_stack; /* Task stack bottom.CNcomment:栈底CNend */
    uint32_t sp; /* Task SP pointer.CNcomment:当前SP.CNend */
    uint32_t curr_used; /* Current task stack usage.CNcomment:当前任务栈使用率CNend */
    uint32_t peak_used; /* Task stack usage peak.CNcomment:栈使用峰值CNend */
    uint32_t overflow_flag; /* Flag that indicates whether a task stack overflow occurs.
    CNcomment:栈溢出标记位CNend */
} task_info_t;

```

说明

在 FreeRTOS 操作系统中，只有下列数据项有效：id、valid、status、priority、bottom_of_stack、overflow_flag，其他数据项可忽略。

命令行代码示例

```
errcode_t diag_cmd_get_task_info(uint16_t cmd_id, void * cmd_param, uint16_t cmd_param_size,
diag_option *option)
{
    uint32_t task_cnt;
    errcode_t ret;
    task_info_t *infs = NULL;
    uapi_unused(cmd_param);
    uapi_unused(cmd_param_size);
    task_cnt = dfx_os_get_task_cnt();
    if (task_cnt == 0) {
        return ERRCODE_FAIL;
    }
    infs = dfx_malloc(0, task_cnt * sizeof(ext_task_info));
    if (infs == TD_NULL) {
        return ERRCODE_FAIL;
    }
    ret = dfx_os_get_all_task_info(infs, task_cnt);
    if (ret != ERRCODE_SUCC) {
        dfx_free(0, infs);
        return ret;
    }
    for (unsigned i = 0; i < task_cnt; i++) {
        task_info_t *inf = &infs[i];
        if (inf->valid) {
            uapi_diag_report_packet(cmd_id, option, (uint8_t *)inf, (uint16_t)sizeof(task_info_t),
true);
        }
    }
    dfx_free(0, infs);
    return ERRCODE_SUCC;
}
```

2 死机诊断

- 2.1 简介
- 2.2 死机信息获取
- 2.3 死机信息查看
- 2.4 常见死机问题定位

2.1 简介

死机诊断是指 BrandyK100 在发生异常时能够记录并输出系统异常信息，后统称死机信息。其中包括死机现场的 PC 地址、返回地址、栈地址、CPU 寄存器信息以及部分内存信息等。用户可根据实际使用场景进行获取并通过分析定位根因，解决死机问题。

BrandyK100 支持以下表 2-1 所示几种方式记录并输出死机信息。

表2-1 死机信息类型说明

类型	说明	依赖	适用场景	所在章节
串口死机信息	死机时在串口直接输出死机信息	串口	适用于未关闭串口的情况。可以获取死机时异常信息、CPU 寄存器信息、函数调用栈信息等。	2.2.1 串口工具界面输出 、 2.3.1 串口死机信息查看
Flash死机文件	死机时在Flash中保存死机信息	Flash	适用于产品阶段，关闭串口输出的情况。可以获取死机时异常信息、CPU 寄存器信息、函数调用栈信息等。	2.2.2 Flash中的死机文件获取 、 2.3.2 Flash中的死

类型	说明	依赖	适用场景	所在章节
				机文件信息查看
Last word	死机时将死机的临终语言输出到 DebugKits 工具	Debug 串口, Debug Kits 工具	适用于未关闭 Debug 串口的情况。可以获取死机时 PC 地址、返回地址、栈地址、CPU 寄存器值等信息。	2.2.3.1 获取 last word 信息 、 2.3.3.1 last word 信息查看
Last dump	死机时将当时的部分内存和寄存器信息输出到 DebugKits 工具	Debug 串口, Debug Kits 工具	适用于未关闭 Debug 串口的情况。可以获取到死机时的大部分内存的原始数据, 以及部分关键寄存器的信息, 现场数据比较丰富, 适用于定位一些踩内存等比较难定位的问题。	2.2.3.2 获取 last dump 信息 、 2.3.3.2 last dump 信息查看
J-Link 调试信息	死机时连接 J-Link 查看现场信息	J-Link	适用于开发初期, 单板需要能够连接 J-Link 调试器。可以获取大量现场数据 (如内存、寄存器、flash 数据等) 。	2.2.4 连接 J-Link 调试器导出 、 2.3.4 连接 J-Link 调试器查看信息

2.2 死机信息获取

本节主要介绍“[表 2-1](#)”中的几种死机信息的获取方式：

- 通过串口工具获取死机打印信息。
- 通过保存在 Flash 中的死机文件获取死机信息。
- 通过 DebugKits 工具获取死机信息。
- 连接 J-Link 调试器获取死机信息。

适用场景：

- 在研发调测场景下，上述四种方法均适用。

- 在外网场景下适用通过 DebugKits 工具或者 Flash 中的死机文件来获取死机信息。

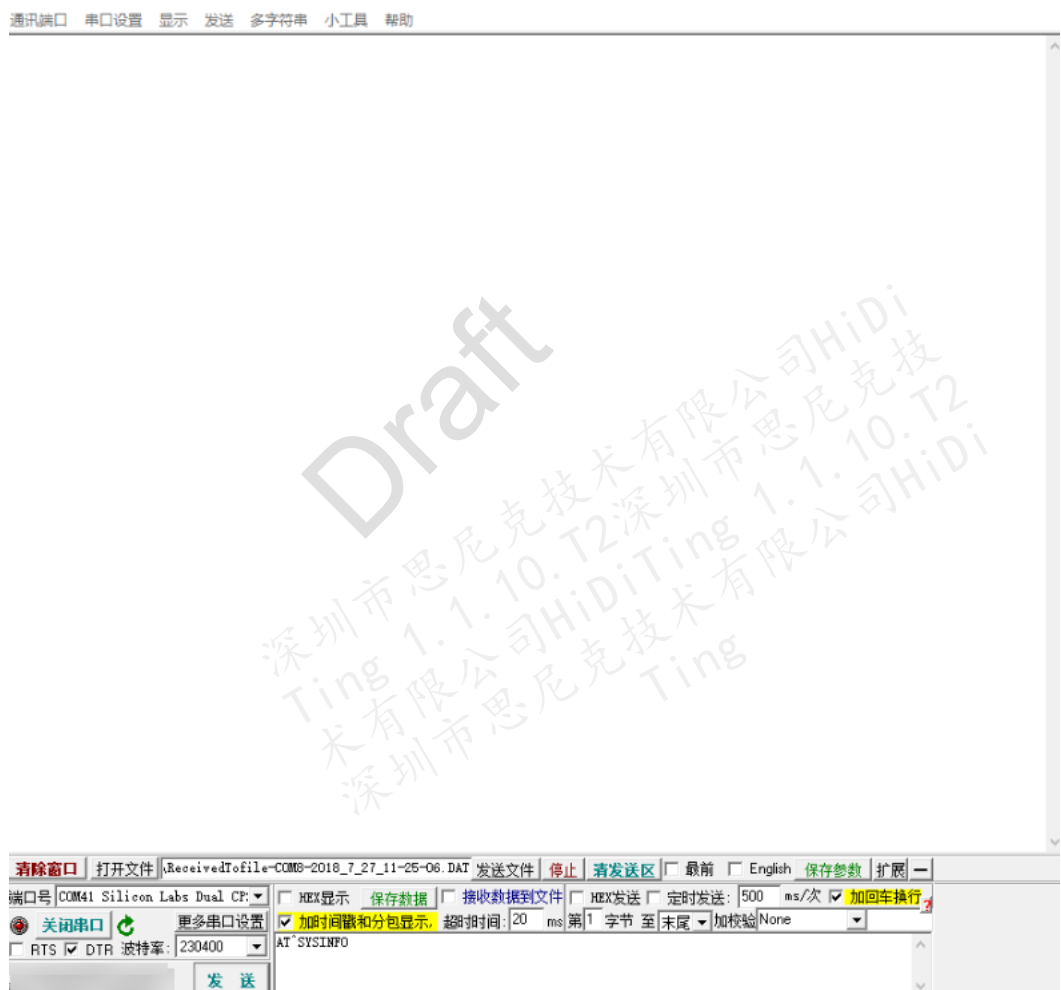
说明

死机信息通过串口打印，要保证死机时单板已连接串口工具。

2.2.1 串口工具界面输出

单板与 PC 通过串口连接之后，打开串口工具界面。选择对应的端口号之后点击打开串口进行连接，波特率选择 115200，如图 2-1 所示。

图2-1 串口连接



当单板与串口工具处于连接状态时，如果单板发生死机，串口工具能够接收到复位前 OS 打印的死机信息。如图 2-2 所示。

图2-2 串口工具中捕获的死机信息

```

task: dfx_msg
thrId: 0x4
type: 0x7
nestCnt: 1
phase: 1
ccause: 0x1
mcause: 0x38000007
mtval: 0xffffffff
gp: 0x2000f2a8
mstatus: 0x80207880
mepc: 0x7048326a
ra: 0x70483314
sp: 0x20037d54
X4 : 0x0
X5 : 0x20037f68
X6 : 0x0
X7 : 0xefbead03
X8 : 0x20037e80
X9 : 0xc8
X10: 0x5079
X11: 0x20037e90
X12: 0x1
X13: 0x20037e90
X14: 0xffffffff
X15: 0x4e
X16: 0xffffffff
X17: 0x0
X18: 0x20037f4c
X19: 0x20037eb4
X20: 0x2002fc14
X21: 0x20037f18
X22: 0x9090909
X23: 0x8080808
X24: 0x7070707
X25: 0x6060606
X26: 0x5050505
X27: 0x4040404
X28: 0x0
X29: 0x24
X30: 0x50000
X31: 0xefbeadde
APP *****dump call stack begin*****
APP call stack 0 — ra = 0x70483314    fp = 0x20037e90
APP call stack 1 — ra = 0x704849f2    fp = 0x20037eb0
APP call stack 2 — ra = 0x704858bc    fp = 0x20037f10
APP call stack 3 — ra = 0x70485a60    fp = 0x20037f30
APP call stack 4 — ra = 0x7048563e    fp = 0x20037f40
APP call stack 5 — ra = 0x70428a5c    fp = 0x20037f70
APP call stack 6 — ra = 0x70401d34    fp = 0x20037f90
APP call stack 7 — ra = 0x70400714    fp = 0xf0f0f0f
APP *****dump call stack end*****

```

2.2.2 Flash 中的死机文件获取

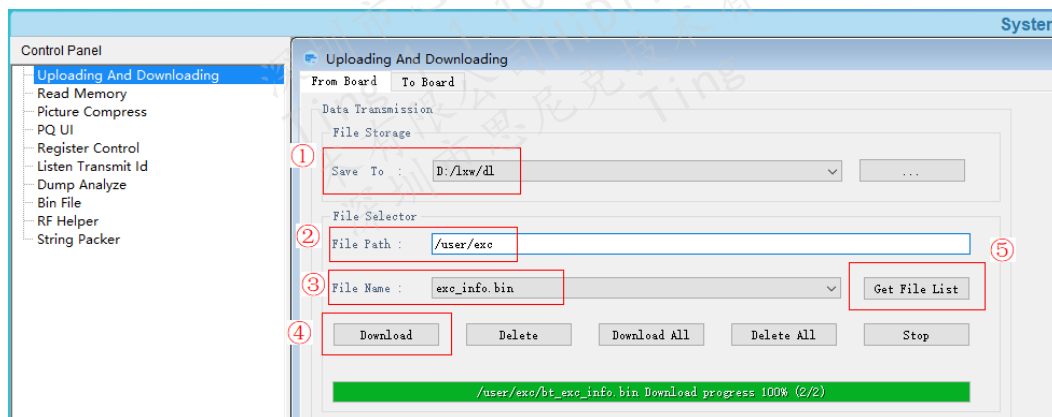
在发生死机异常时，系统会同时将死机信息以死机文件的形式保存在 Flash 文件系统中。

- APP 核死机文件路径：/user/exc/exc_info.bin
- BT 核死机文件路径：/user/exc/bt_exc_info.bin

死机文件可使用 DebugKits 工具导出，步骤如下：

(DebugKits 详细操作方法请参见 “A DebugKits 使用说明” 以及《HiDiTingV100 DebugKits 工具 使用指南》)

图2-3 死机文件导出方法



步骤 1 在标注①处选好死机文件导出存放位置。

步骤 2 在标注②处填写 “/user/exc”。

步骤 3 在标注⑤处点击 “Get File List” 按钮获取该目录下所有文件，在标注③处选择需要下载的死机文件。

步骤 4 在标注④处点击 “Download” 或 “Download All” 按钮，死机文件将下载至标注①选择的目录位置。

----结束

2.2.3 通过 DebugKits 工具获取

在死机故障发生时，如果 DebugKits 处于连接状态，会获取到两种死机信息：

- last word 信息
- last dump 信息

参照 “A DebugKits 使用说明” 中的步骤一~步骤四，打开 DebugKits 工具的消息界面，可以实时观测芯片运行时打印的具体信息，如图 2-4。

当死机发生时消息界面将会打印出 last word 以及 last dump 信息。

图2-4 DebugKits 日志打印

	NO	Category	Time	TimeStamp	Prim	Source	Destination
19	0	Message	12:07:47.112	1970-01-01 08:00:00.000	last_word	-	-
20	0	Message	12:07:49.248	1970-01-01 08:00:00.000	last_dump	-	-
21	0	Message	12:07:49.326	1970-01-01 08:00:00.000	last_dump	-	-
22	0	Message	12:07:49.420	1970-01-01 08:00:00.000	last_dump	-	-
23	0	Message	12:07:49.482	1970-01-01 08:00:00.000	last_dump	-	-
24	0	Message	12:07:49.560	1970-01-01 08:00:00.000	last_dump	-	-
25	0	Message	12:07:49.644	1970-01-01 08:00:00.000	last_dump	-	-
26	0	Message	12:07:49.732	1970-01-01 08:00:00.000	last_dump	-	-
27	0	Message	12:07:49.810	1970-01-01 08:00:00.000	last_dump	-	-
28	0	Message	12:07:49.891	1970-01-01 08:00:00.000	last_dump	-	-
29	0	Message	12:07:49.966	1970-01-01 08:00:00.000	last_dump	-	-
30	0	Message	12:07:50.045	1970-01-01 08:00:00.000	last_dump	-	-
31	0	Message	12:07:50.124	1970-01-01 08:00:00.000	last_dump	-	-
32	0	Message	12:07:50.202	1970-01-01 08:00:00.000	last_dump	-	-
33	0	Message	12:07:50.279	1970-01-01 08:00:00.000	last_dump	-	-
34	0	Message	12:07:50.357	1970-01-01 08:00:00.000	last_dump	-	-
35	0	Message	12:07:50.435	1970-01-01 08:00:00.000	last_dump	-	-
36	0	Message	12:07:50.513	1970-01-01 08:00:00.000	last_dump	-	-
37	0	Message	12:07:50.669	1970-01-01 08:00:00.000	last_dump	-	-
38	0	Message	12:07:50.763	1970-01-01 08:00:00.000	last_dump	-	-
39	0	Message	12:07:50.841	1970-01-01 08:00:00.000	last_dump	-	-
40	0	Message	12:07:50.920	1970-01-01 08:00:00.000	last_dump	-	-
41	0	Message	12:07:50.998	1970-01-01 08:00:00.000	last_dump	-	-
42	0	Message	12:07:51.076	1970-01-01 08:00:00.000	last_dump	-	-

2.2.3.1 获取 last word 信息

last word 信息是死机发生时的临终遗言，其中包括 PC 地址、返回地址、栈地址、CPU 寄存器值等信息。用户可以根据 last word 的具体信息分析错误类型、定位错误原因。

在 DebugKits 的 Message 界面，点击 last word 那一行，即可查看 last word 具体信息，其中，A 核的 last word 和 BT 核的 last word 有所不同，A 核的 last word 信息如图 2-5 所示。BT 核 last word 信息如图 2-6 所示。

图2-5 A 核 last word 信息

0	Message	14:21:37.588	1970-01-01 08:00:00.000	last_word	-	-
rser Pane						
0	1	2	3	4	5	6
0	58	00	00	08	00	00
0	00	00	00	00	00	00
0	00	00	00	00	00	00
6	C0	8C	6C	00	00	00
0	00	00	92	D0	42	70
8	C7	8C	6C	94	78	03
3	AD	B8	EF	00	01	20
9	50	00	00	79	50	00
0	C1	8C	6C	00	00	00
0	00	00	58	C7	8C	6C
F	9A	03	00	00	01	20
8	C6	8C	6C	4C	C1	8C
5	A5	A5	A5	A5	A5	A5
5	A5	A5	A5	A5	A5	A5
5	A5	A5	A5	A5	A5	A5
4	00	00	00	00	05	00
6	C0	8C	6C	00	00	00
C	DC	42	70	00	00	00
E	AD	B8	EF	00	00	00
7	00	00	00	00	00	00
Name						
Value(Hex)						
Type						
diag_last_word_ind_t						
stack_limit						
0x00005800						
td_u32						
fault_type						
0x00000008						
td_u32						
fault_reason						
0x00000000						
td_u32						
address						
0x00000000						
td_u32						
reg_value[0]						
0x00000000						
td_u32[32]						
reg_value[1]						
0x00000000						
td_u32[32]						
reg_value[2]						
0x6C8C096						
td_u32[32]						
reg_value[3]						
0x00000000						
td_u32[32]						
reg_value[4]						
0x00000000						
td_u32[32]						
reg_value[5]						
0x7042d92						
td_u32[32]						
reg_value[6]						
0x6C8C7e8						
td_u32[32]						
reg_value[7]						
0x00037b94						
td_u32[32]						
reg_value[8]						
0xfefbead03						
td_u32[32]						
reg_value[9]						
0x20010000						
td_u32[32]						
reg_value[10]						
0x00005079						
td_u32[32]						
reg_value[11]						
0x00005079						
td_u32[32]						
reg_value[12]						
0x6C8C110						
td_u32[32]						
reg_value[13]						
0x00000000						
td_u32[32]						
reg_value[14]						
0x6C8C110						
td_u32[32]						
reg_value[15]						
0x00000000						
td_u32[32]						
reg_value[16]						
0x0000004e						
td_u32[32]						
reg_value[17]						
0x6C8C7e8						
td_u32[32]						
reg_value[18]						
0x00039a0f						
td_u32[32]						
reg_value[19]						
0x20010000						
td_u32[32]						
reg_value[20]						
0x6C8C6b8						
td_u32[32]						
reg_value[21]						
0x6C8C14c						
td_u32[32]						
reg_value[22]						
0xa5a5a5a5						
td_u32[32]						
reg_value[23]						
0xa5a5a5a5						
td_u32[32]						
reg_value[24]						
0xa5a5a5a5						
td_u32[32]						
reg_value[25]						
0xa5a5a5a5						
td_u32[32]						
reg_value[26]						
0xa5a5a5a5						
td_u32[32]						
reg_value[27]						
0xa5a5a5a5						
td_u32[32]						
reg_value[28]						
0xa5a5a5a5						
td_u32[32]						
reg_value[29]						
0x00000000						
td_u32[32]						
reg_value[30]						
0x00000024						
td_u32[32]						
reg_value[31]						
0x00005000						
td_u32[32]						
psp_value						
0x6C8C096						
td_u32						
lr_value						
0x00000000						
td_u32						

图2-6 BT 核 last word 信息

	NO	Category	Time	TimeStamp	Prio	Source	Destination
73421	74656	System	10:22:22.008	2024-04-21 09:04:59.000	[INFO][SLEEP]bt_sleepstore_glbctrl(0x270c2d) @bt_pmu.c...	BT	0x00000000
73422	74657	System	10:22:22.008	2024-04-21 09:04:59.000	[INFO][SLEEP]bt_sleep_wakeup_fincnt(241) jlp_period(1...	BT	0x00000000
73423	74658	System	10:22:22.008	2024-04-21 09:04:59.000	[INFO][SLEEP]wake_up_end_clk(2032655) fincnt(543) glbctrl(0x...	BT	0x00000000
73424	74659	System	10:22:22.008	2024-04-21 09:04:59.000	unknown	PLATFORM	0x00000000
73425	74660	System	10:22:22.008	2024-04-21 09:04:59.000	unknown	PLATFORM	0x00000000
73426	74661	System	10:22:42.078	2024-04-21 09:05:19.000	unknown	PLATFORM	0x00000000
73427	74662	System	10:22:42.080	2024-04-21 09:05:19.000	unknown	PLATFORM	0x00000000
73428	74663	System	10:22:42.080	2024-04-21 09:05:19.000	unknown	PLATFORM	0x00000000
73429	74664	System	10:22:47.312	2024-04-21 09:05:24.000	EXCEPTION_LAST_RUN_INFO	BT	0x00000000
73430	74665	System	10:22:51.978	2024-04-21 09:05:29.000	unknown	0x0000000b	0x00000000
73431	74666	System	10:22:51.978	2024-04-21 09:05:29.000	unknown	0x0000000b	0x00000000
73432	74667	System	10:23:02.156	2024-04-21 09:05:39.000	unknown	PLATFORM	0x00000000

Parser Pane

0 1 2 3 4 5

00 00 00 00 0C 00 0
7E 10 03 00 00 00 0
9E 0A 01 00 90 FC 0
0C 00 00 80 00 00 7
00 00 00 00 00 00 0
00 00 00 00 00 00 0
00 00 00 00 00 00 0
00 00 7B 02 0C 00 0
0C 00 00 80 90 FC 0
8A C4 00 00 00 00 0
0C 00 00 80 7E 10 0
00 00 00 00 7E 10 0
00 00 00 00

AirTree

stack-limit0x0 (0)

fault-type0x8000000c (2147483660)

fault-reason0x3107e (200830)

address0x0 (0)

reg-value

UINT320x10a9e (68254)

UINT320x0a690 (457872)

UINT320x8000000c (2147483660)

UINT320x27d0000 (41746432)

UINT320x0 (0)

UINT320x0 (0)

UINT320x0 (0)

UINT320x0 (0)

UINT320x0 (0)

UINT320x0 (0)

UINT320x27d0000 (41746432)

UINT320x8000000c (2147483660)

UINT320x8000000c (2147483660)

psp-value0x6fc90 (457872)

lr-value0x4ba (50314)

pc-value0x0 (0)

psps-value0x8000000c (2147483660)

primask-value0x3107e (200830)

fault-mask-value0x0 (0)

bseprpri-value0x3107e (200830)

control-value0x0 (0)

A 核和 BT 核的 last word 信息差异在于：

- 在 message 界面中名称不同：A 核名称为 last_word，BT 核名称为 EXCEPTION_LAST_RUN_INFO。
- 内容中 reg_value 的个数不同和对应的含义不同：A 核 reg_value 为 32 个，BT 核 reg_value 为 13 个。其对应的含义详见“2.3.3.1 last word 信息查看”章节。

2.2.3.2 获取 last dump 信息

last dump 信息是在死机故障发生时，将单板中的部分内存和寄存器信息通过串口发送到 PC 端，通过 DebugKits 工具生成 bin 文件存储到的安装目录中的 DumpInfo 子目录之下，具体参考图 2-7。

用户可以通过解析这些文件来定位死机原因。解析这些文件的方法请参见“3 Last Dump 解析”。

2025-06-05

19

图2-7 last dump 生成文件

电脑 > SystemDisk (C:) > DebugKits > DumpInfo > DumpInfo_1				
名称	修改日期	类型	大小	
APP_DTCM_ORIGIN.bin	2023/3/16 12:23	BIN 文件	448 KB	
APP_ITCM_ORIGIN.bin	2023/3/16 12:23	BIN 文件	192 KB	
BCPU_TRACE_MEM_REGION.bin	2023/3/16 12:24	BIN 文件	8 KB	
BT_RAM_ORIGIN_APP_MAPPING.bin	2023/3/16 12:24	BIN 文件	576 KB	
MCPU_TRACE_MEM_REGION.bin	2023/3/16 12:23	BIN 文件	8 KB	
PMU1_CTL.bin	2023/3/16 12:24	BIN 文件	2 KB	
PMU2_CTL.bin	2023/3/16 12:24	BIN 文件	2 KB	
SHARED_MEM.bin	2023/3/16 12:23	BIN 文件	96 KB	
ULP_AON_CTL.bin	2023/3/16 12:24	BIN 文件	2 KB	

说明

将内存 dump 到 DebugKits 中需要一定时间 (Liteos: 1~2 min, FreeRtos: 4~5 min), 因此如果需要使用 last dump 分析死机信息, 注意不要在发生死机故障后立即复位, 否则 last dump 信息可能无法完整获取。

2.2.4 连接 J-Link 调试器导出

在允许连接 JTAG 调试器的场景下, 用户可连接劳特巴赫或 J-Link 仿真器等工具进一步获取死机现场的相关信息。以下主要介绍通过 J-Link 仿真器获取死机相关信息的过程。

在条件允许的情况下, 建议开启死机不复位功能进行死机定位。死机现场时, 连接 J-Link, 可以查看“[2.2.1 串口工具界面输出](#)”小节所展示的信息, 还可查询内存信息、栈信息、外设寄存器、维测变量等更丰富的数据。

使用 J-Link 仿真器调试需先连接单板的 20PIN 排针, 具体排针位置以实际产品为准。

硬件连接后, 双击工具包 (sdk\tools\bin\jlink_tool\brandy) 中的 **Commandline_brandy_mcpu.bat** 脚本, 脚本将自动进行连接 (其他核的连接操作可参照此方法进行)。

图2-8 J-Link 连接 MCPPU 示意图

```
管理员: MCPPU Command Line
Connecting to J-Link via USB...O.K.
Firmware: J-Link V11 compiled Apr 27 2041, 16:36:21
Hardware version: V11.00
S/N: 50120677
License(s): GDB, JFlash, FlashDL, RDI, FlashBP
VTref=1.896V
Device "RISC-V" selected.

Connecting to target via SWD
ConfigTargetSettings() start
ConfigTargetSettings() end
InitTarget() start
J-Link script: Connect to Core 1
InitTarget() end
Found SW-DP with ID 0x20000477
RISC-V behind DAP detected
DPIDR: 0x20000477
AP[1] (AHB-AP) specified by user as debug AP.
AP map scan skipped.
ROMBASE: 0x00000000
Core base addr.: 0x00000000 (assumed)
Memory access:
  Via system bus: No
  Via ProgBuf: Yes (3 ProgBuf entries)
DataBuf: 1 entries
  autoexec[0] implemented: Yes
Detected: RV32 core
CSR access via abs. commands: No
Temp. halted CPU for NumHWBP detection
HW instruction/data BPs: 6
Support set/clr BPs while running: No
HW data BPs trigger before execution of inst
RISC-V identified.
J-Link>
```

须知

注意 RISC-V 架构处理器需要使用 V10 及以上版本的 J-Link 仿真器。

2.3 死机信息查看

对于“[2.2 死机信息获取](#)”中描述的几种方式获取的死机信息，本节分别介绍它们的查看方式和定位方法。

2.3.1 串口死机信息查看

在死机故障发生时，一般会在串口输出“[2.2.1 串口工具界面输出](#)”中描述的死机信息。其中包含几个部分：异常信息汇总、CPU 寄存器信息、函数调用栈信息。

2.3.1.1 异常信息汇总

表2-2 异常信息汇总

成员	说明
task	死机的任务名称。
thrdPid	死机的任务 ID。
type	死机类型，type = mcause & 0xFFFF。

2.3.1.2 CPU 寄存器信息

表2-3 死机相关 CPU 寄存器说明

成员	说明
mepc	机器异常程序计数器。当发生异常时，mepc 指向导致异常的指令；对于中断，mepc 指向中断处理后应该恢复的位置。
mstatus	机器状态寄存器。
mtval	机器陷入寄存器。保存地址异常中出错的地址或者发生指令异常的指令本身，对于其他错误，其值为零。
mcause	机器异常寄存器。保存目前异常或者中断的原因，通过查询表 2-4 得到目前异常或者中断的类型。表格中的异常码 = mcause & 0xFFFF。
ccause	ccause 为 mcause 的补充说明，对于某些异常通过读取 ccause 寄存器的内容可以进一步明确异常类型。

图2-9 死机相关 CPU 寄存器说明

寄存器	ABI名字	描述	保存者
x0	zero（零）	硬件连线0	—
x1	ra	返回地址	调用者
x2	sp	栈指针	被调用者
x3	gp	全局指针	—
x4	tp	线程指针	—
x5-7	t0-2	临时变量	调用者
x8	s0/fp	保存的寄存器/帧指针	被调用者
x9	s1	保存的寄存器	被调用者
x10-11	a0-1	函数参数/返回值	调用者
x12-17	a2-7	函数参数	调用者
x18-27	s2-11	保存的寄存器	被调用者
x28-31	t3-6	临时变量	调用者
f0-7	ft0-7	FP临时变量	调用者
f8-9	fs0-1	FP保存的寄存器	被调用者
f10-11	fa0-1	FP参数/返回值	调用者
f12-17	fa2-7	FP参数	调用者
f18-27	fs2-11	FP保存的寄存器	被调用者
f28-31	ft8-11	FP临时变量	调用者

表2-4 mcause&ccause 异常描述表

异常码	mcause 异常描述	ccause 异常描述
0x0	Instruction address misaligned	Not available
0x1	Instruction access fault	Memory map region access fault
0x2	Illegal instruction	AXIM error response
0x3	Breakpoint	AHBM error response
0x4	Load address misaligned	Crossing PMP entries
0x5	Load access fault	System register access fault

异常码	mcause 异常描述	ccause 异常描述
0x6	Store/AMO address misaligned	No PMP entry matched
0x7	Store/AMO access fault	PMP access fault
0x8	Environment call from U-mode	CMO access fault
0x9	Environment call from S-mode	CSR access fault
0xA	Reserved	LDM/STMIA instruction
0xB	Environment call from M-mode	ITCM write access fault
0xC	Instruction page fault	Not available
0xD	Load page fault	Not available
0xE	Reserved	Not available
0xF	Store/AMO page fault	Not available
0xFFF	NMI interrupt	Not available

2.3.1.3 函数调用栈信息

函数调用栈 call stack 会显示与异常相关的所有函数调用指令。用户可以根据函数调用栈检查异常发生时函数调用的上下文定位。其中 call back 0 为栈顶函数，其对应的 ra 为栈顶函数的地址，以此类推。

用户可以到 “output/brandy/acore/brandy-xxx/application.lst” 中找到对应的函数。

2.3.2 Flash 中的死机文件信息查看

按照 “2.2.2 Flash 中的死机文件获取” 中的描述，从 Flash 中导出的死机文件为二进制文件，需使用死机信息解析脚本（dump_parser）解析后查看。

死机信息解析脚本支持在 Windows 和 Linux 环境下运行，可解析从 Flash 中读出的 A 核和 BT 核死机信息。

- Linux 运行方式：进入脚本目录给予脚本可执行权限并运行下列命令，结果如图 2-10 所示。

```
chmod +x run.sh
```

```
./run.sh <死机信息文件路径> [lst 文件路径（可选）]
```

图2-10 Linux 环境运行死机脚本工具

```

~/codehub/dump_tool/dump_parsers$ sh run.sh exc_info.bin application.lst
Available last word types:
1. melody-a
2. melody-bt
3. brandy-a-liteos
4. brandy-a-freertos
5. brandy-bt
Input number for last word or q for quit (num/q): 4
Last word info:
task_name: at_cmd
uw_exc_type: 0x00000007 (7)
ret: 0x00000000 (0)
mepc: 0x703f84ee (1883210990)
mstatus: 0x80207880 (2149611648)
mtval: 0x00000000 (0)
mcause: 0x00000000 (0)
ra: 0x2002c328 (537051944)
sp: 0x70405b24 (1883265828)
gp: 0xa5a5a5a5 (2779096485)
tp: 0x2003f4e0 (537130208)
t0: 0x0000000a (10)
t1: 0xffffffff (4294967295)
t2: 0x200244b8 (537019576)
s0: 0x00000005 (5)
s1: 0x0000001d (29)
a0: 0x80000000 (2147483648)
a1: 0x2002c324 (537051940)
a2: 0x2002c328 (537051944)
a3: 0x20024581 (537019777)
a4: 0x00000000 (0)
a5: 0x00000005 (5)
a6: 0x00000001 (1)
a7: 0x704d8260 (1884127840)
s2: 0x2001c38c (536986508)
s3: 0x0000000b (11)
s4: 0x2001c000 (536985600)
s5: 0x2001c48c (536986764)
s6: 0x2001c38c (536986508)
s7: 0x00000047 (71)
s8: 0xa5a5a5a5 (2779096485)
s9: 0xa5a5a5a5 (2779096485)
s10: 0xa5a5a5a5 (2779096485)
s11: 0xa5a5a5a5 (2779096485)
t3: 0x00000000 (0)
t4: 0x00000000 (0)
t5: 0x00000000 (0)
t6: 0x00000000 (0)
reg0: 0x00000000 (0)
reg1: 0x00000000 (0)
reg2: 0x00000000 (0)
reg3: 0x00000000 (0)
stack_size: 0x00000002 (2)
stack:
[0]: ra: 0x704b79b4 fp: 0x2003f530 <app_fs_rw_test_handle>
[1]: ra: 0x00024ec2 fp: 0xa5a5a5a5 <at_cmd_task_process_handler>
===== finished! =====
~/codehub/dump_tool/dump_parsers$

```

- Windows 运行方式：在脚本目录下打开命令提示符，运行下列命令即可开始解析，结果如图 2-11 所示。

run.bat <死机信息文件路径> [lst 文件路径 (可选)]

图2-11 Windows 环境运行死机脚本工具

```

D:\dump_parser>run.bat exc_info.bin application.lst
Available last word types:
1. melody-a
2. melody-bt
3. brandy-a-liteos
4. brandy-a-freertos
5. brandy-bt
Input number for last word or q for quit (num/q): 4
Last word info:
task_name: at_cmd
uw_exc_type: 0x00000007 (7)
ret: 0x00000000 (0)
mepc: 0x703f84ee (1883210990)
mstatus: 0x80207880 (2149611648)
mtval: 0x00000000 (0)
mcause: 0x00000000 (0)
ra: 0x2002c328 (537051944)
sp: 0x70405b24 (1883265828)
gp: 0xa5a5a5a5 (2779096485)
tp: 0x2003f4e0 (537130208)
t0: 0x0000000a (10)
t1: 0xffffffff (4294967295)
t2: 0x200244b8 (537019576)
s0: 0x00000005 (5)
s1: 0x0000001d (29)
a0: 0x80000000 (2147483648)
a1: 0x2002c324 (537051940)
a2: 0x2002c328 (537051944)
a3: 0x20024581 (537019777)
a4: 0x00000000 (0)
a5: 0x00000005 (5)
a6: 0x00000001 (1)
a7: 0x704d8260 (1884127840)
s2: 0x2001c38c (536986508)
s3: 0x0000000b (11)
s4: 0x2001c000 (536985600)
s5: 0x2001c48c (536986764)
s6: 0x2001c38c (536986508)
s7: 0x00000047 (71)
s8: 0xa5a5a5a5 (2779096485)
s9: 0xa5a5a5a5 (2779096485)
s10: 0xa5a5a5a5 (2779096485)
s11: 0xa5a5a5a5 (2779096485)
t3: 0x00000000 (0)
t4: 0x00000000 (0)
t5: 0x00000000 (0)
t6: 0x00000000 (0)
reg0: 0x00000000 (0)
reg1: 0x00000000 (0)
reg2: 0x00000000 (0)
reg3: 0x00000000 (0)
stack_size: 0x00000002 (2)
stack:
[0]: ra: 0x704b79b4 fp: 0x2003f530 <app_fs_rw_test_handle>
[1]: ra: 0x00024ec2 fp: 0xa5a5a5a5 <at_cmd_task_process_handler>
===== finished! =====

```

Flash 死机文件解析后的具体内容以及含义，A 核的死机信息可参考“[2.3.1 串口死机信息查看](#)”；BT 核的死机信息，可参考“[2.3.3.1 last word 信息查看](#)”。

说明

使用中的常见问题：

- 运行死机文件解析脚本需要安装 Python 3.8 及以上版本，并将安装路径配置到系统环境变量。建议可在执行解析脚本之前先运行 python 命令判断是否满足条件。如下所示：

```
D:\Tools\死机解析脚本\dump_parser_v1.5>python
Python 3.8.4rc1 (tags/v3.8.4rc1:6c38841, Jun 30 2020, 15:17:30) [MSC v.1924 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> quit()
```

- 运行死机文件解析脚本需要安装 pycparser 库，如运行时提示找不到 pycparser，可使用“pip install pycparser”命令安装 pycparser 库。

2.3.3 通过 DebugKits 获取的死机信息查看

2.3.3.1 last word 信息查看

如“[2.2.3.1-获取 last word 信息](#)”中描述，在发生死机故障时，会将 last word 信息发送到 DebugKits 工具上。

last word 上报的内容及含义如表 2-5 所示（具体含义中括号里面的为代称）。

表2-5 last word 内容

变量名称	具体含义
stack_limit	系统栈大小。
fault_type	错误类型（mcause）。其中 NMI interrupt 在 A 核为 0x80000FFF，在 BT 核为 0x8000000C。
fault_address	错误地址，属性值无意义。
fault_reason	错误原因，属性值无意义。
reg_value	保存寄存器值。
psp_value	栈指针（sp）。
lr_value	返回地址（ra）。
pc_value	发生错误的指令地址（mepc）。
psps_value	全局指针（gp）。

变量名称	具体含义
primask_value	异常状态寄存器（mstatus）。
fault_mask_value	CPU 访问异常地址或异常值（mtval）。
bserpri_value	自定义异常状态寄存器（ccause）。
control_value	CPU 访问异常地址或异常值（mtval）。

对于 last word 内容中保存的寄存器的值（reg_value），其个数和具体含义 A 核和 BT 核有所不同。

具体含义如表 2-6 和表 2-7 所示。

表2-6 A 核 last word 中 reg_value 成员含义

成员名称	成员含义
reg_value[0]	无意义
reg_value[1]	返回地址（ra）
reg_value[2]	栈指针（sp）
reg_value[3]	全局指针（gp）
reg_value[4]	线程指针（tp）
reg_value[5]	临时寄存器（t0）
reg_value[6]	临时寄存器（t1）
reg_value[7]	临时寄存器（t2）
reg_value[8]	被调函数需要保存用的寄存器/调用栈的帧指针（s0）
reg_value[9]	被调函数需要保存用的寄存器（s1）
reg_value[10]	函数参数/返回值（a0）
reg_value[11]	函数参数/返回值（a1）
reg_value[12]	函数参数（a2）
reg_value[13]	函数参数（a3）

成员名称	成员含义
reg_value[14]	函数参数 (a4)
reg_value[15]	函数参数 (a5)
reg_value[16]	函数参数 (a6)
reg_value[17]	函数参数 (a7)
reg_value[18]	被调函数需要保存用的寄存器 (s2)
reg_value[19]	被调函数需要保存用的寄存器 (s3)
reg_value[20]	被调函数需要保存用的寄存器 (s4)
reg_value[21]	被调函数需要保存用的寄存器 (s5)
reg_value[22]	被调函数需要保存用的寄存器 (s6)
reg_value[23]	被调函数需要保存用的寄存器 (s7)
reg_value[24]	被调函数需要保存用的寄存器 (s8)
reg_value[25]	被调函数需要保存用的寄存器 (s9)
reg_value[26]	被调函数需要保存用的寄存器 (s10)
reg_value[27]	被调函数需要保存用的寄存器 (s11)
reg_value[28]	临时寄存器 (t3)
reg_value[29]	临时寄存器 (t4)
reg_value[30]	临时寄存器 (t5)
reg_value[31]	临时寄存器 (t6)

表2-7 BT 核 last word 中 reg_value 成员含义

成员名称	成员含义
reg_value[0]	函数参数/返回值 (a0)
reg_value[1]	函数参数/返回值 (a1)

成员名称	成员含义
reg_value[2]	函数参数 (a2)
reg_value[3]	函数参数 (a3)
reg_value[4]	函数参数 (a4)
reg_value[5]	函数参数 (a5)
reg_value[6]	函数参数 (a6)
reg_value[7]	函数参数 (a7)
reg_value[8]	被调函数需要保存用的寄存器/调用栈的帧指针 (s0)
reg_value[9]	被调函数需要保存用的寄存器 (s1)
reg_value[10]	被调函数需要保存用的寄存器 (s10)
reg_value[11]	被调函数需要保存用的寄存器 (s11)
reg_value[12]	被调函数需要保存用的寄存器 (s2)

2.3.3.2 last dump 信息查看

如“2.2.3.2-获取 last dump 信息”中描述，当死机故障发生时，同时会将当前的内存 dump 到 DebugKits 保存下来，并可使用相关工具做进一步解析。详细使用方法请参见“3 Last Dump 解析”。

在解析生成的“memory_result.txt”中搜索“死机”可查询到死机信息，如图 2-12 所示。

图2-12 last dump 中的死机信息

```

-----content of 死机信息:-----
g_exc_buff_addr{
  .ccause=0x1
  .mcause=0x38000007
  .mtval=0xffffffff
  .gp=0x0
  .task_context.mstatus=0x80207880
  .task_context.mepc=0x70485a66
  .task_context.tp=0x0
  .task_context.sp=0x20032834
  .task_context.s11=0xa5a5a5a5
  .task_context.s10=0xa5a5a5a5
  .task_context.s9=0xa5a5a5a5
  .task_context.s8=0xa5a5a5a5
  .task_context.s7=0xa5a5a5a5
  .task_context.s6=0xa5a5a5a5
  .task_context.s5=0x200328e8
  .task_context.s4=0x2002bc58
  .task_context.s3=0x20032884
  .task_context.s2=0x2003291c
  .task_context.s1=0xc8
  .task_context.s0=0x20032850
  .task_context.t6=0xefbeadde
  .task_context.t5=0x50000
  .task_context.t4=0x24
  .task_context.t3=0x0
  .task_context.a7=0x0
  .task_context.a6=0xffffffff
  .task_context.a5=0x4e
  .task_context.a4=0xffffffff
  .task_context.a3=0x20032860
  .task_context.a2=0x1
  .task_context.a1=0x20032860
  .task_context.a0=0x5079
  .task_context.t2=0xefbead03
  .task_context.t1=0x0
  .task_context.t0=0x20032938
  .task_context.ra=0x70485b14
}
[BACK Trace][addr=0x200328bc][val=0x704030e0]xQueueReceive
[BACK Trace][addr=0x200328dc][val=0x704882d8]diag_pkt_msg_proc
[BACK Trace][addr=0x200328ec][val=0x0001a38c]osal_msg_queue_read_copy
[BACK Trace][addr=0x200328fc][val=0x70487eae]diag_msg_proc
[BACK Trace][addr=0x2003290c][val=0x70429e24]msg_process_thread
[BACK Trace][addr=0x2003293c][val=0x0001f26e]xPortHwiEnable
[BACK Trace][addr=0x20032e08][val=0x0001f912]uapi_pm_init
[BACK Trace][addr=0x20032e34][val=0x0001f92a]pm_dev_ostimer_ioctl
[BACK Trace][addr=0x20032e60][val=0x0001f96c]pm_dev_wdt_ioctl
[BACK Trace][addr=0x20032eb8][val=0x0001f9b6]pm_dev_rtc_ioctl
[BACK Trace][addr=0x20032ee4][val=0x0001f9b6]pm_dev_rtc_ioctl
[BACK Trace][addr=0x200337a4][val=0x0001d030]vHwiYield
[BACK Trace][addr=0x2003382c][val=0x70402b66]xQueueSemaphoreTake

```

其中的内容与串口中的死机信息类似（如图 2-2 所示），不同的是，last dump 的死机信息中函数调用栈中的地址会自动找到对应的函数。

2.3.4 连接 J-Link 调试器查看信息

本小节主要介绍在死机现场使用 J-Link 调试器导出死机信息的过程。

通过“2.2.4 连接 J-Link 调试器导出”的方法连接 J-Link 后，可使用表 2-8 中的命令查看当前的状态和信息，如：寄存器、内存、flash 中的数据。

表2-8 J-Link 常用命令

命令	说明
con/connect	连接。
h/halt	暂停，停止。
g/go	继续，运行。
mem32 mem16 mem8	I/O 读 32bit: mem32 <Addr>, <NumItems> (hex) (Addr 表示内存地址 NumItems 表示从 Addr 开始连续读几个 32bit) I/O 读 16bit: mem16 <Addr>, <NumItems> (hex) I/O 读 8bit: mem8 <Addr>, <NumItems> (hex)
w4 w2 w1	I/O 写 4Byte: w4 <Addr>, <Data> (hex) I/O 写 2Byte: w2 <Addr>, <Data> (hex) I/O 写 1Byte: w1 <Addr>, <Data> (hex)
readcsr	读 riscv csr 寄存器: ReadCSR <RegIndex>
writcsr	写 riscv csr 寄存器: WriteCSR <RegIndex>,<Value>

查询 I/O 信息

步骤 1 查询 CPU 寄存器信息，如图 2-13 所示。

图2-13 查询 CPU 寄存器值

```

J-Link>h
PC = 0040005A
X0 = 00000000, X1 = 00400056, X2 = 10049130, X3 = 10000520
X4 = 00000000, X5 = 00000008, X6 = 00100160, X7 = 1C1C1C1C
X8 = 10000650, X9 = 1000065C, X10 = 80006088, X11 = 00000000
X12 = 0000001C, X13 = 00000001, X14 = 70003000, X15 = 78001000
X16 = 10049058, X17 = 14141414, X18 = 0D0D0D0D, X19 = 0C0C0C0C
X20 = 0B0B0B0B, X21 = 0A0A0A0A, X22 = 09090909, X23 = 08080808
X24 = 07070707, X25 = 06060606, X26 = 05050505, X27 = 04040404
X28 = 13131313, X29 = 12121212, X30 = 11111111, X31 = 10101010
J-Link>readcsr 0x341
CSR 0x0341: 0x0010134A
J-Link>_

```

步骤 2 查询内存信息，获取维测变量值或普通变量值。

1. 在 “\output\brandy\acore\brandy-xxx\xxx.nm” 中获取 nm 文件，在 nm 文件中获取想要查询的变量地址如图 2-14 中 “g_exception_dump_callback” 变量的地址为 0x20025950。

图2-14 变量地址

```

g_exception_dump_callback|20025950|...b...|...OBJECT|00000004|...|.bss

```

2. 通过 J-Link 获取状态信息，如图 2-15 所示。

图2-15 变量值

```

J-Link>mem32 0x20025950 1
20025950 = 602512FC

```

----结束

说明

以上地址信息仅作为演示使用，查询步骤供用户参考使用。

2.4 常见死机问题定位

2.4.1 看门狗重启问题定位

看门狗死机导致的重启问题，大概率为代码中存在死循环（包含次数很大的循环）或某个业务一直被执行（例如灌包），使得 IDLE 任务无法得到调度，规定时间内未进行踢狗造成的重启。通常可以通过表 2-9 中的信息并结合 lst 文件，确定具体的死机位置。

表2-9 看门狗死机关键信息

成员	说明
type/mcause	看门狗死机，死机类型为 0xFFFF。
mepc	通过 mepc 确定看门狗到期时的 pc。根据概率推测可知，该代码基本为死循环中会调用的代码。
返回地址 (ra) 和栈	如果 mepc 在通用函数中，例如 memcpy 函数中，可以通过 ra 和栈内容进一步确认函数调用流程。

2.4.2 CPU 异常触发重启问题定位

通过 fault_type 值的含义的描述内容，可确认 CPU 异常触发重启具体为哪种死机问题。

表2-10 CPU 异常的 fault_type 含义

fault_type 错误	说明
Instruction address misaligned	取址地址不对齐，riscv 要求指令双字节对齐位置。
Instruction access fault	取址地址异常。
Illegal instruction	非法指令。
Load address misaligned	取数据的目标地址不对齐。
Load access fault	取数据的目的地址是禁止进行读操作的地址。
Store/AMO address misaligned	存储数据目标地址不对齐。
Store/AMO access fault	存储数据的目的地址是禁止进行写操作的地址。

2.4.2.1 Store 或 Load 异常

表2-11 store 或 load 异常关键信息

成员	说明
mepc	通过 mepc 确定异常的语句。
返回地址 (ra) 和栈	如果 mepc 在通用函数中, 如 memcpy 中, 可以通过 ra 和栈内容进一步确认函数调用流程。
mtval	通过 mtval 确认访问的异常地址。
寄存器值	如 mepc 指向下面语句时, 可以确认 a1 值是否合法。 - lw a0,0(a1): 将 a0 存储到 a1 指向的位置。 - sw a0,0(a1): 从 a1 指向位置获取内容到 a0。

说明

- 由于存储存在异步性, 存储异常时 mepc 指向的语句可能不是异常语句。
- RISCv lb、lh、lw、ld、sb、sh、sw、sd 指令要求访问地址字节对齐。
- lb (load byte): 对齐要求为 1 字节。
- lh (load halfword): 对齐要求为 2 字节。
- lw (load word): 对齐要求为 4 字节。
- ld (load doubleword): 对齐要求为 8 字节。
- sb (store byte): 对齐要求为 1 字节。
- sh (store halfword): 对齐要求为 2 字节。
- sw (store word): 对齐要求为 4 字节。
- sd (store doubleword): 对齐要求为 8 字节。

因此, 对于 load 和 store 指令, 被访问的地址不满足对应数据长度的对齐要求, 将会引发地址对齐异常, 建议客户在定义全局变量时, 使用 `__attribute__((aligned(8)))` 进行对齐。

2.4.2.2 取指异常

取指异常通常为 PC 跑飞造成的。首先可以确认 mepc 的范围是否正常, 通过 ra 和调用栈分析可以确认代码跑飞的位置。

造成代码跑飞主要原因如下:

- 存储函数跳转地址的钩子变量被踩，导致函数跳转到非法地址。
- 栈被踩，导致函数无法正确返回上一级函数而跑飞。
- 代码段本身被踩。

Draft

3 Last Dump 解析

3.1 概述

3.2 功能描述

3.1 概述

Last Dump 解析功能是指将死机发生时导出的内存数据的内容进行解析，将.bin 文件解析为可读性较好的文本文件，为问题定位提供参考信息。

3.2 功能描述

Last Dump 解析功能主要包括两个部分：

- 第一部分主要功能是解析 DWAEF 文件的 “.debug_info” 段，将对应的 tag 解析生成 python 格式的 class 文件，同时解析符号表，获取全局变量信息。此步骤当前集成在版本构建流程中。
- 第二部分主要功能是将第一步生成的文件作为输入，解析死机 dump 文件，生成文本信息。此步骤当前集成在 DebugKits 工具中。

3.2.1 场景说明

针对死机问题定位信息少，问题定位困难等场景，提供 Last Dump 解析功能，为死机问题定位提供更多的参考信息。

3.2.2 工作流程

步骤 1 死机 dump 文件获取，请参考“2.2.3.2 获取 last dump 信息”。其中，DebugKits 工具导出的 last dump 生成文件就是待解析的死机文件（Last Dump 功能是否开启通过 DUMP_MEM_SUPPORT、DUMP_REG_SUPPORT、SUPPORT_DFX_EXCEPTION 这三个宏控制）。

步骤 2 编译构建生成解析工具。上文已经介绍了解析功能的第一步集成在版本构建流程中，Last Dump 解析前需要先进行版本编译构建获取 parse_tool 解析工具。

1. 获取源码进行版本编译，编译环境需要支持 python3，Last Dump 解析功能所需脚本和中间文件在版本编译过程中生成，编译过程中会提示信息：Build parse tool success。
2. 编译结果确认，在 output 目录 “/output/brandy/acore/<target-name>” 下生成 “application.nm” 文件、“application.info” 文件以及 “parse_tool” 文件夹，其中 “parse_tool” 文件夹下包括如下文件。

└─ auto_class.py 此文件在编译过程中生成，内存解析脚本需要 import 的 python 模块。

└─ auto_struct.txt database 路径下 “mss_cmd_db.xml” 文件内结构体自动生成结果，可拷贝到 Debugkits 工具的 database 路径下，用于 Debugkits 工具解析日志。

└─ global.txt 此文件在编译过程中生成，内存解析时需要导入的全局变量信息。

└─ config.py 内存解析脚本配置文件。

└─ parse_basic.py 内存解析脚本。

└─ parse_elf.py 内存解析脚本。

└─ parse_freertos.py FreeRTOS 系统解析脚本。

└─ parse_liteos.py Liteos 系统解析脚本。

└─ parse_main_phase1.py 内存解析脚本的第一阶段入口，已集成在编译构建过程中，主要目的时生成上面的 “auto_class.py”、“auto_struct.txt” 以及 “global.txt” 文件。

└─ parse_main_liteos206_phase2.py Liteos206 版本内存解析脚本的第二阶段入口，对应 Liteos206 版本的解析，在 DebugKits 工具界面调用，用于将全内存.bin 文件解析为用户可见的文本信息。

└─ parse_main_liteos207_phase2.py Liteos207 版本内存解析脚本的第二阶段入口，对应 Liteos207 版本的解析，在 DebugKits 工具界面调用，用于将全内存.bin 文件解析为用户可见的文本信息。

└─ parse_main_freertos_phase2.py FreeRTOS 版本内存解析脚本的第二阶段入口，对应 FreeRTOS 版本的解析，在 DebugKits 工具界面调用，用于将全内存.bin 文件解析为用户可见的文本信息。

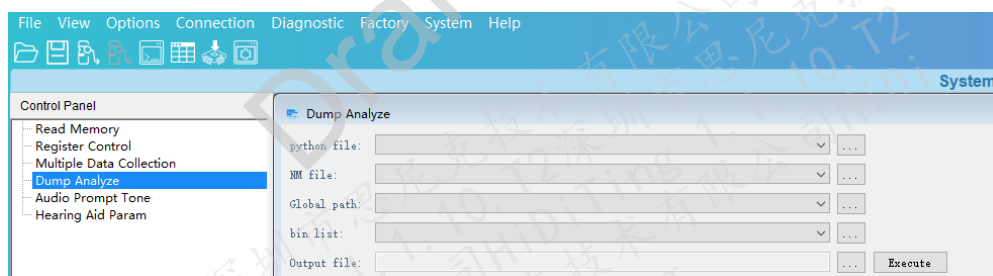
└─ parse_print_global_var.py 内存解析脚本。

└─ xml_main.py 内存解析脚本。

步骤 3 执行 Last Dump 解析。在执行 Last Dump 解析之前，确保运行解析工具的 PC 上已安装 python3 和 DebugKits 工具。

1. 在 DebugKits 工具的 System 界面选取 “DumpAnalyzs” 选项，在对应的选项上选取相应的路径；如图 3-1 所示（注：也可以将编译后的 parse_tool 文件夹，“application.nm” 文件及 last dump 内存导出文件拷贝到本地 PC 上进行处理，工具选取对应路径即可）。

图3-1 DebugKits 工具 Dump 解析界面



- python file 是解析脚本的路径，需要用户选取，例如
“/output/brandy/acore/<target-name>/parse_tool/
parse_main_****_phase2.py”。根据用户使用的 OS 不同，Liteos 版本使用
parse_main_liteos207_phase2.py，freertos 版本使用
parse_main_freertos_phase2.py。
- NM file 为内存解析脚本的输入文件，在版本构建时通过编译命令解析 elf 文件生成，路径：/output/brandy/acore/<target-name>/application.nm。
- Global path 为内存解析时需要导入的全局变量信息，在版本构建时通过解析脚本的第一步生成，路径：/output/brandy/acore/<target-name>/parse_tool/global.txt。

- bin list 为待解析的内存文件路径（即通过 DebugKits 工具导出的内存 bin 文件的存储路径：例如：D:\DebugKits\DumpInfo\DumpInfo_1），只需要选取路径即可，工具会自动加载该路径下相关的.bin 文件。
 - Output file 为解析结果文件存放路径，需要用户选取，例如“D:\DebugKits\DumpInfo\DumpInfo_1\memory_result.txt”。
2. 单击 Dump Analyze 界面的 “Execute” 按钮，正常情况下 DebugKits 工具界面会显示解析结果，并且在用户选取的 Output file 路径下会生成 “memory_result.txt” 解析结果如图 3-2 所示，如果没有显示结果，则参考 “3.2.3 异常处理” 的介绍进行处理。

图3-2 Last Dump 解析结果文件

APP_DTCM_ORIGIN.bin	2023/11/4 18:06	BIN 文件	448 KB
APP_ITCM_ORIGIN.bin	2023/11/4 18:06	BIN 文件	192 KB
DumpInfo.ini	2023/11/4 18:09	配置设置	1 KB
MCPU_TRACE_MEM_REGION.bin	2023/11/4 18:06	BIN 文件	8 KB
memory_result.txt	2023/11/4 18:10	文本文档	1,229 KB
PMU1_CTL.bin	2023/11/4 18:09	BIN 文件	2 KB
PMU2_CTL.bin	2023/11/4 18:09	BIN 文件	2 KB
PSRAM_HEAP.bin	2023/11/4 18:09	BIN 文件	7,168 KB
SHARED_MEM.bin	2023/11/4 18:06	BIN 文件	96 KB
ULP_AON_CTL.bin	2023/11/4 18:09	BIN 文件	2 KB

3. 分析解析结果。
- 打开 “memory_result.txt” 文件可以查询到当前的中断信息、任务信息、消息队列、信号量、全局变量、死机信息等。

图3-3 Last Dump 解析结果片段

```

-----content of 中断信息:-----
HalTickEntry[0x16040][call_cnt=22323]
bt_int0_handler[0x1446c][call_cnt=0]
dsp_int0_handler[0x1444c][call_cnt=0]
irq_uarth0_handler[0x116a6][call_cnt=69]
irq_uartl0_handler[0x116a2][call_cnt=0]
pmu_wakeup_handler[0x15322][call_cnt=0]
pmu_sleep_handler[0x153aa][call_cnt=0]
rtc_irq_handler[0x15726][call_cnt=0]
pmu_cmu_interrupt_handler[0x15476][call_cnt=0]
-----content of 调度信息汇总:-----
说明:taskLockCnt锁任务嵌套次数
g_tickCount=2232311,g_sysClock=32000000,g_tickPerSecond=
100,g_cycle2NsScale=0.000000
schedule{
    .taskSortLink.sortLink=0x2000a490
    .taskSortLink.cursor=0x0
    .taskSortLink.reserved=0x0
    .swtmrSortLink.sortLink=0x2000a464
    .swtmrSortLink.cursor=0x0
    .swtmrSortLink.reserved=0x0
    .expiryList.head=0x19
    .expiryList.tail=0x19
    .expiryList.free=0x32
    .expiryList.buf=0x2000bc70
    .idleTaskId=0x1
    .taskLockCnt=0x0
    .swtmrHandlerQueue=0x0
    .swtmrTaskId=0x0
    .schedFlag=0x0
}

```

----结束

3.2.3 异常处理

1. 如果执行 Last Dump 解析后 DebugKits 工具没有显示解析结果，首先需要确认“application.nm”文件、“application.info”文件、“parse_tool”脚本及导出的“.bin”文件是否出自同一个软件版本。不同版本的输入不匹配可能会导致异常。
2. DebugKits 工具导入待解析的.bin 文件时选取的是整个目录，要保证 DebugKits 工具配置文件“\DebugKits\Source-Datas\bin\config\config.ini”中的配置与 DumpInfo 文件夹下导出文件的地址及大小一致，否则也会导致解析异常。

可以将“config.ini”中的 DumpInfo 配置项与源码

“middleware\chips\brandy\dfx\last_dump_adapt.c”中的 g_mem_dump_info 数组中的元素进行对比。如不一致，则要按照 g_mem_dump_info 数组的定义修改“config.ini”中的 DumpInfo 配置项。

例如图 3-4 中的 APP_ITCM_ORIGIN_StartAddr 的值应该等于图 3-5 中的 APP_ITCM_ORIGIN 的值, APP_ITCM_ORIGIN_Length 的值应该等于 APP_ITCM_LENGTH 的值。

图3-4 config.ini 中配置信息

```
[DumpInfo]
APP_ITCM_ORIGIN_StartAddr=0x10000
APP_ITCM_ORIGIN_Length=0x30000
APP_DTCM_ORIGIN_StartAddr=0x20000000
APP_DTCM_ORIGIN_Length=0x70000
SHARED_MEM_StartAddr=0x87000000
SHARED_MEM_Length=0x18000
MCPU_TRACE_MEM_REGION_StartAddr=0x52006000
MCPU_TRACE_MEM_REGION_Length=0x1FF8
BT_RAM_ORIGIN_APP_MAPPING_StartAddr=0xA6000000
BT_RAM_ORIGIN_APP_MAPPING_Length=0x90000
BCPU_TRACE_MEM_REGION_StartAddr=0x5900E000
BCPU_TRACE_MEM_REGION_Length=0x1FF8
PSRAM_HEAP_StartAddr=0x6c8c8b80
PSRAM_HEAP_Length=0x937480
```

图3-5 g_mem_dump_info 数组

```
static diag_mem_dump_t g_mem_dump_info[] = {
    { "APP_ITCM_ORIGIN", APP_ITCM_ORIGIN, APP_ITCM_LENGTH },
    { "APP_DTCM_ORIGIN", APP_DTCM_ORIGIN, APP_DTCM_LENGTH },
#ifdef UNSUPPORT_OTHER_MEM
    { "SHARED_MEM", SHARED_MEM_START, SHARED_MEM_LENGTH },
    { "MCPU_TRACE_MEM_REGION", MCPU_TRACE_MEM_REGION_START, CPU_TRACE_MEM_REGION_LENGTH },
    { "BT_RAM_ORIGIN_APP_MAPPING", BT_RAM_ORIGIN_APP_MAPPING, BT_RAM_ORIGIN_APP_MAPPING_LENGTH },
    { "BCPU_TRACE_MEM_REGION", BCPU_TRACE_MEM_REGION_START, CPU_TRACE_MEM_REGION_LENGTH },
#endif
#ifdef __FREERTOS__
    { "PSRAM_HEAP", PSRAM_HEAP_ADDR, PSRAM_HEAP_LENGTH },
#endif /* __FREERTOS__ */
#ifdef UNSUPPORT_OTHER_MEM
};
```

3. 上述两点都排查无误后, 需要手工执行脚本进行问题定位。

方法如下:

- a. 打开 DebugKits 工具安装目录下的 “log.txt”, 找到工具调用解析脚本的命令。图 3-6 中 cmd 后面双引号内的内容即执行 Last Dump 解析的命令, 需要注意删除其中的转义字符 “\”。

图3-6 DebugKits 工具调用解析脚本的命令

```

Debug: File:(..\..\..\Code\hsoui\System\DumpAnalysWin.cpp) Line:(219)
[StartPythonExe::run]----cmd: " python
D:/Demo/parse_tool/parse_main_liteos_phase2.py --rm_file
D:/Demo/parse_tool/application.rm --bin_list "D:/DumpInfo/DumpInfo_
1/APP_DTCM_ORIGIN.bin|536870912|458752|D:/DumpInfo/DumpInfo_
1/APP_ITCM_ORIGIN.bin|65536|196608|D:/DumpInfo/DumpInfo_
1/BCPU_TRACE_MEM_REGION.bin|1493229568|8184|D:/DumpInfo/DumpInfo_
1/BT_RAM_ORIGIN_APP_MAPPING.bin|2785017856|589824|D:/DumpInfo/DumpInfo_
1/MCPU_TRACE_MEM_REGION.bin|1375756288|8184|D:/DumpInfo/DumpInfo_1/PMU1
_CTL.bin|0|0|D:/DumpInfo/DumpInfo_1/PMU2_CTL.bin|0|0|D:/DumpInfo/DumpInfo_
1/SHARED_MEM.bin|2264924160|98304|D:/DumpInfo/DumpInfo_1/ULP_AON_CTL.bin|0|
0| " --result D:/DumpInfo/DumpInfo_1/memory_result.txt --global
D:/Demo/parse_tool/global.txt" (2023-03-16 10:17:28.897)

```

- b. 将命令拷贝到 cmd 窗口执行，可以看到解析脚本的执行结果。图 3-7 中是正确执行的结果。如果出现错误，可以根据错误提示，进一步定位问题。

图3-7 Last Dump 解析在命令行窗口执行结果

```

D:\>python D:/Demo/parse_tool/parse_main_liteos_phase2.py --rm_file D:/Demo/parse_tool/application.rm --bin_list "D:/DumpInfo/DumpInfo_1/APP_DTCM_ORIGIN.bin|536870912|458752|D:/DumpInfo/DumpInfo_1/APP_ITCM_ORIGIN.bin|65536|196608|D:/DumpInfo/DumpInfo_1/BCPU_TRACE_MEM_REGION.bin|1493229568|8184|D:/DumpInfo/DumpInfo_1/BT_RAM_ORIGIN_APP_MAPPING.bin|2785017856|589824|D:/DumpInfo/DumpInfo_1/MCPU_TRACE_MEM_REGION.bin|1375756288|8184|D:/DumpInfo/DumpInfo_1/PMU1_CTL.bin|0|0|D:/DumpInfo/DumpInfo_1/PMU2_CTL.bin|0|0|D:/DumpInfo/DumpInfo_1/SHARED_MEM.bin|2264924160|98304|D:/DumpInfo/DumpInfo_1/ULP_AON_CTL.bin|0|0| " --result D:/DumpInfo/DumpInfo_1/memory_result.txt --global D:/Demo/parse_tool/global.txt
Running python at C:\Program Files\Python38\python.exe version:3.8.5
args:
Namespace(bin_list='D:/DumpInfo/DumpInfo_1/APP_DTCM_ORIGIN.bin|536870912|458752|D:/DumpInfo/DumpInfo_1/APP_ITCM_ORIGIN.bin|65536|196608|D:/DumpInfo/DumpInfo_1/BCPU_TRACE_MEM_REGION.bin|1493229568|8184|D:/DumpInfo/DumpInfo_1/BT_RAM_ORIGIN_APP_MAPPING.bin|2785017856|589824|D:/DumpInfo/DumpInfo_1/MCPU_TRACE_MEM_REGION.bin|1375756288|8184|D:/DumpInfo/DumpInfo_1/PMU1_CTL.bin|0|0|D:/DumpInfo/DumpInfo_1/PMU2_CTL.bin|0|0|D:/DumpInfo/DumpInfo_1/SHARED_MEM.bin|2264924160|98304|D:/DumpInfo/DumpInfo_1/ULP_AON_CTL.bin|0|0|', rm_file='D:/Demo/parse_tool/application.rm', result='D:/DumpInfo/DumpInfo_1/memory_result.txt')
parse_print_global_var end

```

4 内存 DFX 功能

4.1 概述

4.2 功能描述

4.1 概述

内存 DFX 功能主要用于内存调优分析，主要包括系统内存占用分析、各任务内存占用分析、任务栈使用率分析以及内存泄漏检测。

4.2 功能描述

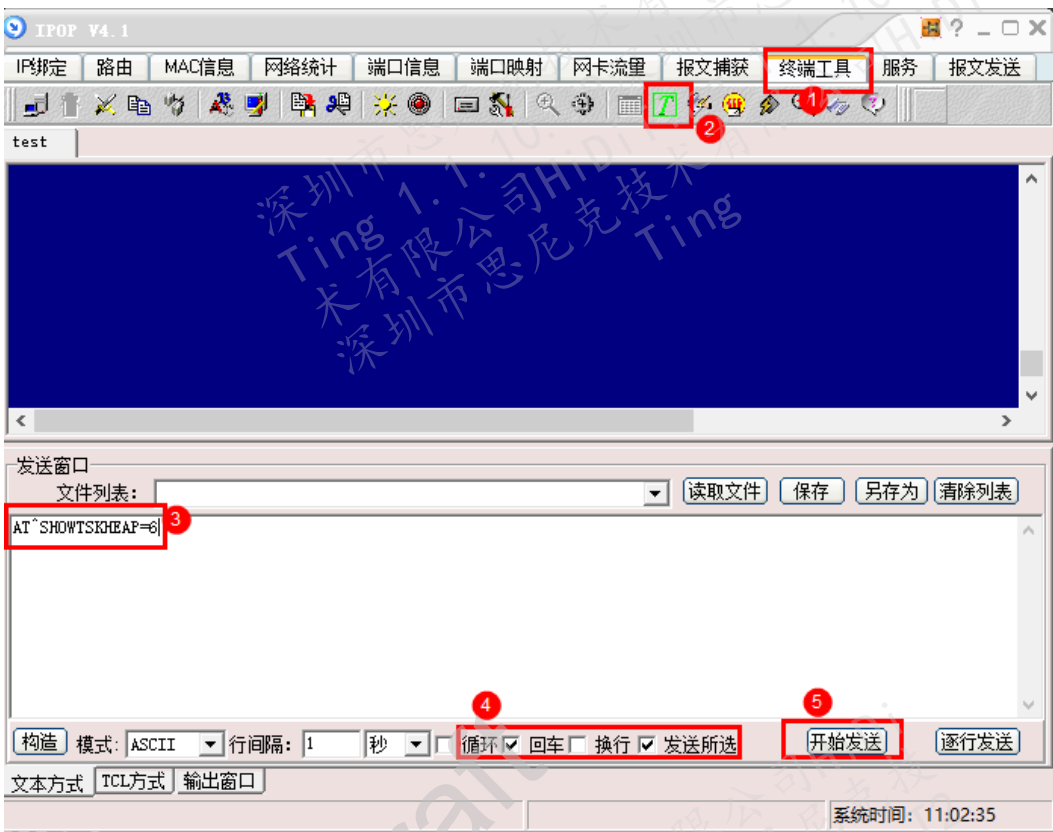
须知

所有命令在执行时均会**关中断**，因此可能导致消息队列满等异常，建议尽量避免在压测场景下使用“AT^SHOWALLTSKHEAP、AT^SHOWMEMINFO”指令，这两个指令均可以用其他指令组合后替代。

4.2.1 查询指定任务内存占用情况

步骤 1 使用任意串口工具下发“AT^SHOWTSKHEAP=<taskId>”命令，taskId 的获取请参见“4.2.6 查询任务信息”，以 IPOP 工具为例，如图 1 串口命令所示。

图4-1 串口命令



步骤 2 命令下发后可以得到四个信息。

- ①当前任务占用内存的详细信息，size 表示一个内存节点的大小，addr 表示该内存节点的起始地址，callerRA 表示该内存节点申请时的下一条指令地址。
- ②当前任务占用的非 slab 内存总量。
- ③当前任务占用的 slab 内存总量。
- ④当前系统中因为内存节点申请而带来的总内存开销。

具体内容如图 2 命令执行结果所示。

图4-2 命令执行结果

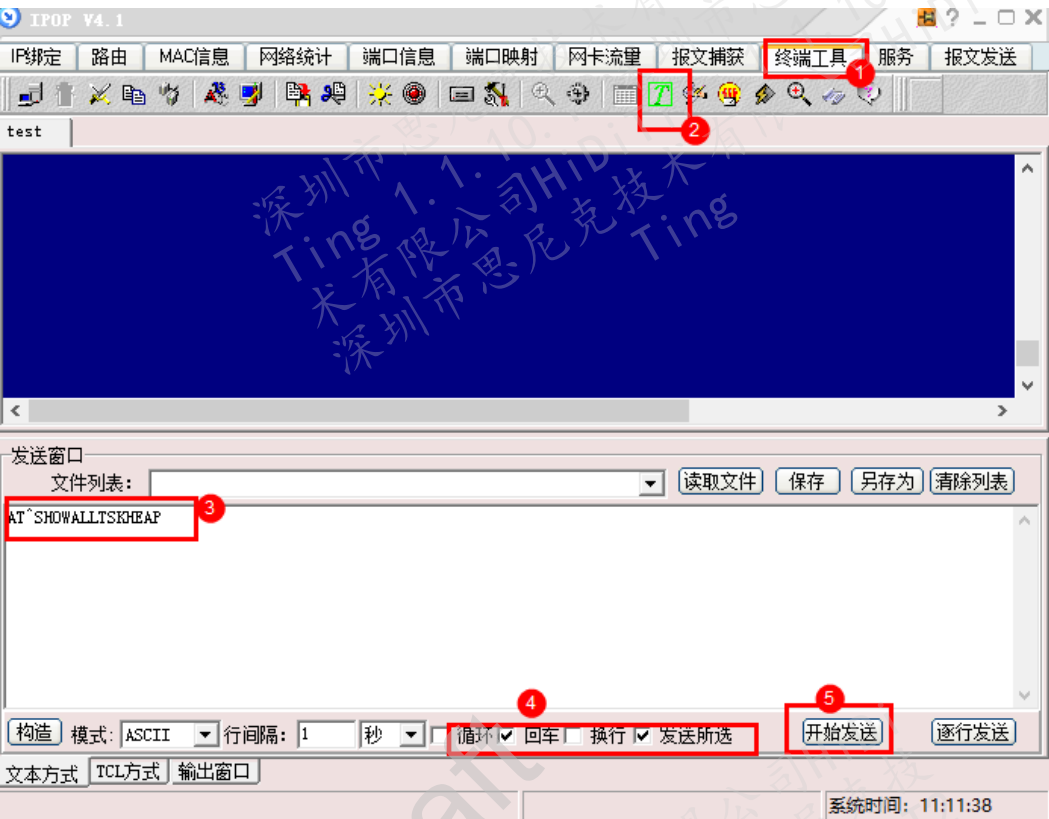
```
Task:app(taskId=06) malloc details:
mem[001]: size=00000040 Bytes, addr=0x6c91ac7c, callerRA=0x702dc1aa
mem[002]: size=00000040 Bytes, addr=0x6c91ac54, callerRA=0x702e930c
mem[003]: size=00000032 Bytes, addr=0x6c91ac34, callerRA=0x702dc1aa
mem[004]: size=00000032 Bytes, addr=0x6c91ac14, callerRA=0x702dc1aa
mem[005]: size=00000032 Bytes, addr=0x6c91abf4, callerRA=0x702dc1aa
mem[006]: size=00000032 Bytes, addr=0x6c91abd4, callerRA=0x702dc1aa
mem[007]: size=00000032 Bytes, addr=0x6c91abb4, callerRA=0x702dc1aa
mem[008]: size=00000028 Bytes, addr=0x6c91ab98, callerRA=0x702dc1aa
mem[009]: size=00000032 Bytes, addr=0x6c91ab78, callerRA=0x702dc1aa
mem[010]: size=00000060 Bytes, addr=0x6c91ab3c, callerRA=0x702dc1aa
mem[011]: size=00000112 Bytes, addr=0x6c91aacc, callerRA=0x702dc1aa
mem[012]: size=00000032 Bytes, addr=0x6c91aaac, callerRA=0x702dc1aa
mem[013]: size=00000060 Bytes, addr=0x6c91aa70, callerRA=0x702dc1aa
mem[014]: size=00000048 Bytes, addr=0x6c91aa40, callerRA=0x702dc1aa
mem[015]: size=00000036 Bytes, addr=0x6c91aa1c, callerRA=0x702dc1aa
mem[016]: size=00000100 Bytes, addr=0x6c91a9b8, callerRA=0x702dc1aa
mem[017]: size=00000072 Bytes, addr=0x6c91a970, callerRA=0x702dc1aa
mem[018]: size=00000028 Bytes, addr=0x6c91a954, callerRA=0x702dc1aa
mem[019]: size=00000032 Bytes, addr=0x6c91a934, callerRA=0x702dc1aa
mem[020]: size=00000072 Bytes, addr=0x6c91a8ec, callerRA=0x702dc1aa
mem[021]: size=00000028 Bytes, addr=0x6c91a8d0, callerRA=0x702dc1aa
mem[022]: size=00000032 Bytes, addr=0x6c91a8b0, callerRA=0x702dc1aa
mem[023]: size=00000072 Bytes, addr=0x6c91a868, callerRA=0x702dc1aa
mem[024]: size=00000028 Bytes, addr=0x6c91a84c, callerRA=0x702dc1aa
mem[025]: size=00000032 Bytes, addr=0x6c91a82c, callerRA=0x702e930c
mem[026]: size=00000072 Bytes, addr=0x6c91a7e4, callerRA=0x70397a06
mem[027]: size=00000036 Bytes, addr=0x6c919e48, callerRA=0x702dc1aa
Current total pool alloc size=1252 Bytes
Current total slab alloc size=0 Bytes
Current total cost size=20160 Bytes
```

----结束

4.2.2 查询当前所有任务内存占用情况

步骤 1 使用任意串口工具下发“AT+SHOWALLTSKHEAP”命令，以 IPOP 工具为例，如图 4-3 所示。

图4-3 串口命令



步骤 2 命令下发后可以得到当前系统内所有任务的四个信息。

- ①当前任务占用内存的详细信息，size 表示一个内存节点的大小，addr 表示该内存节点的起始地址，callerRA 表示该内存节点申请时的下一条指令地址。
- ②当前任务占用的非 slab 内存总量。
- ③当前任务占用的 slab 内存总量。
- ④当前系统中因为内存节点申请而带来的总内存开销。

具内容如图 4-4 所示。

图4-4 命令执行结果

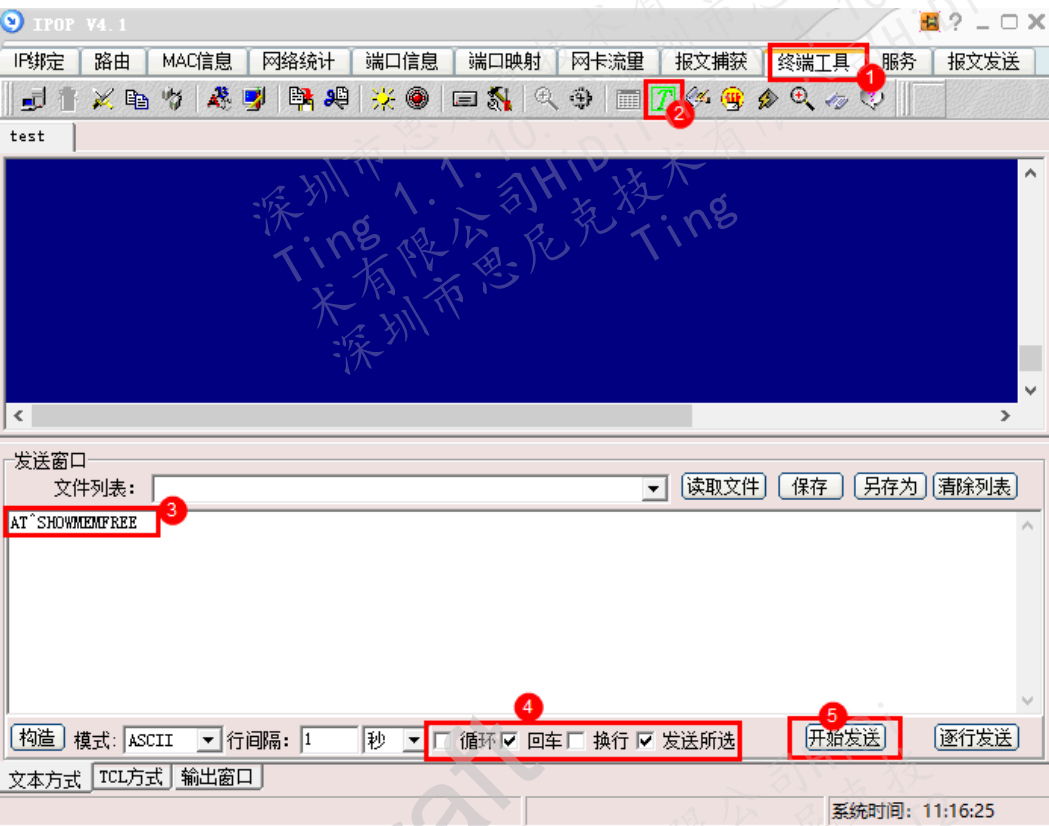
```
Task:app(taskId=06) malloc details:
mem[001]: size=00000040 Bytes, addr=0x6c91ac7c, callerRA=0x702dc1aa
mem[002]: size=00000040 Bytes, addr=0x6c91ac54, callerRA=0x702e930c
mem[003]: size=00000032 Bytes, addr=0x6c91ac34, callerRA=0x702dc1aa
mem[004]: size=00000032 Bytes, addr=0x6c91ac14, callerRA=0x702dc1aa
mem[005]: size=00000032 Bytes, addr=0x6c91abf4, callerRA=0x702dc1aa
mem[006]: size=00000032 Bytes, addr=0x6c91abd4, callerRA=0x702dc1aa
mem[007]: size=00000032 Bytes, addr=0x6c91abb4, callerRA=0x702dc1aa
mem[008]: size=00000028 Bytes, addr=0x6c91ab98, callerRA=0x702dc1aa
mem[009]: size=00000032 Bytes, addr=0x6c91ab78, callerRA=0x702dc1aa
mem[010]: size=00000060 Bytes, addr=0x6c91ab3c, callerRA=0x702dc1aa
mem[011]: size=00000112 Bytes, addr=0x6c91aacc, callerRA=0x702dc1aa
mem[012]: size=00000032 Bytes, addr=0x6c91aaac, callerRA=0x702dc1aa
mem[013]: size=00000060 Bytes, addr=0x6c91aa70, callerRA=0x702dc1aa
mem[014]: size=00000048 Bytes, addr=0x6c91aa40, callerRA=0x702dc1aa
mem[015]: size=00000036 Bytes, addr=0x6c91aa1c, callerRA=0x702dc1aa
mem[016]: size=00000100 Bytes, addr=0x6c91a9b8, callerRA=0x702dc1aa
mem[017]: size=00000072 Bytes, addr=0x6c91a970, callerRA=0x702dc1aa
mem[018]: size=00000028 Bytes, addr=0x6c91a954, callerRA=0x702dc1aa
mem[019]: size=00000032 Bytes, addr=0x6c91a934, callerRA=0x702dc1aa
mem[020]: size=00000072 Bytes, addr=0x6c91a8ec, callerRA=0x702dc1aa
mem[021]: size=00000028 Bytes, addr=0x6c91a8d0, callerRA=0x702dc1aa
mem[022]: size=00000032 Bytes, addr=0x6c91a8b0, callerRA=0x702dc1aa
mem[023]: size=00000072 Bytes, addr=0x6c91a868, callerRA=0x702dc1aa
mem[024]: size=00000028 Bytes, addr=0x6c91a84c, callerRA=0x702dc1aa
mem[025]: size=00000032 Bytes, addr=0x6c91a82c, callerRA=0x702e930c
mem[026]: size=00000072 Bytes, addr=0x6c91a7e4, callerRA=0x70397a06
mem[027]: size=00000036 Bytes, addr=0x6c919e48, callerRA=0x702dc1aa
Current total pool alloc size=1252 Bytes
Current total slab alloc size=0 Bytes
Current total cost size=20160 Bytes
```

----结束

4.2.3 查询当前所有空闲内存信息

步骤 1 使用任意串口工具下发“AT^SHOWMEMFREE”命令，以 IPOP 工具为例，如图 4-5 所示。

图4-5 串口命令

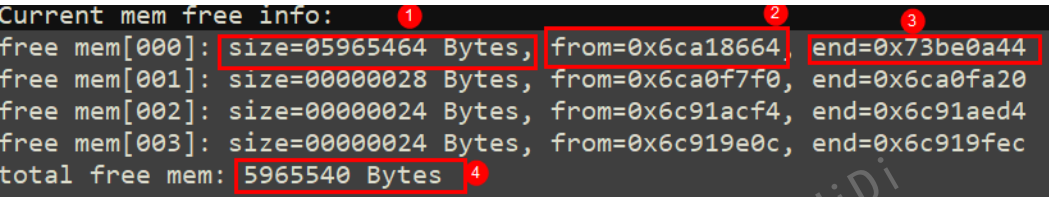


步骤 2 命令下发后可以得到当前系统空闲内存的四个信息：

- ①size：空闲内存块的大小。
- ②from：该空闲块的起始地址。
- ③end：该空闲块的结束地址。
- ④当前总内存剩余量。

具体内容如图 4-6 所示。

图4-6 命令执行结果

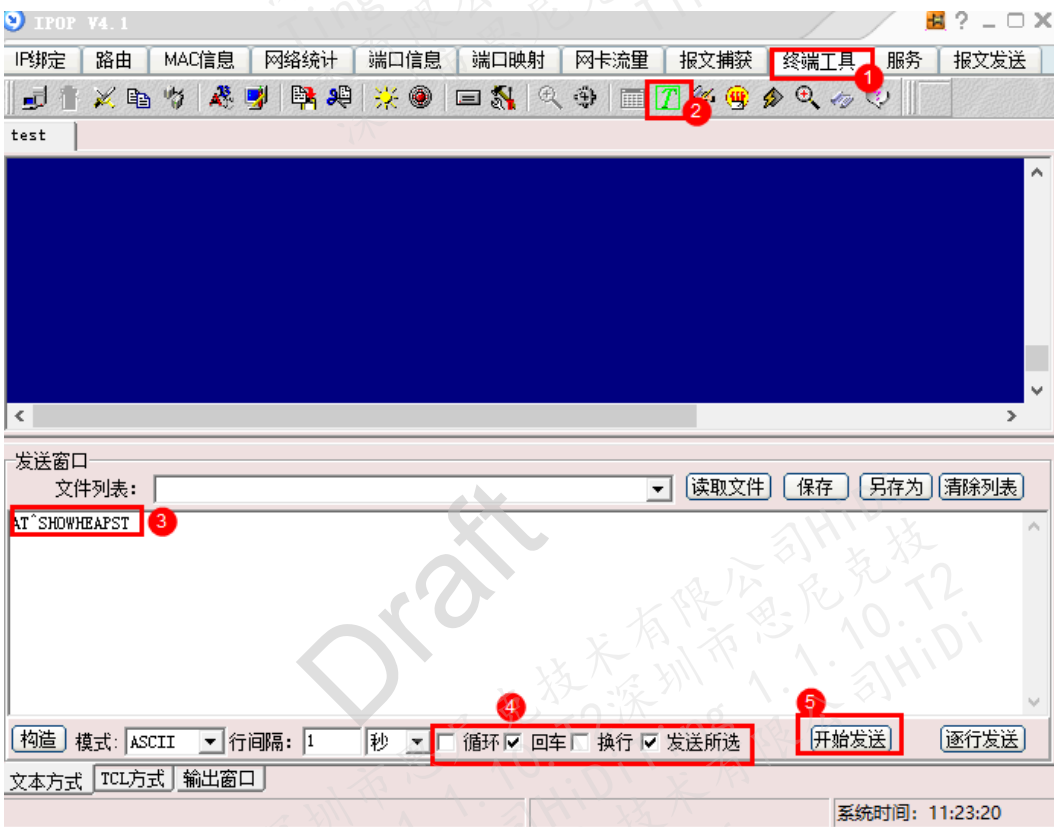


----结束

4.2.4 查询当前内存统计信息

步骤 1 使用任意串口工具下发“AT^SHOWHEAPST”命令，以 IPOP 工具为例，如图 4-7 所示。

图4-7 串口命令



步骤 2 命令下发后可以得到三类信息：

- ①当前内存的使用信息，total 表示内存的总量，used 表示当前已使用的内存量，current free 表示当前剩余内存量，peak usage 表示峰值内存使用量，peak free 表示峰值内存剩余量。
- ②当前所有任务内存分配信息，current malloc 表示截止统计时任务依旧持有的内存量，peak malloc 表示截止统计时任务所持有内存的最大值。
- ③非任务所占用的内存，比如中断等。

具体内容如图 4-8 所示。

图4-8 命令执行结果

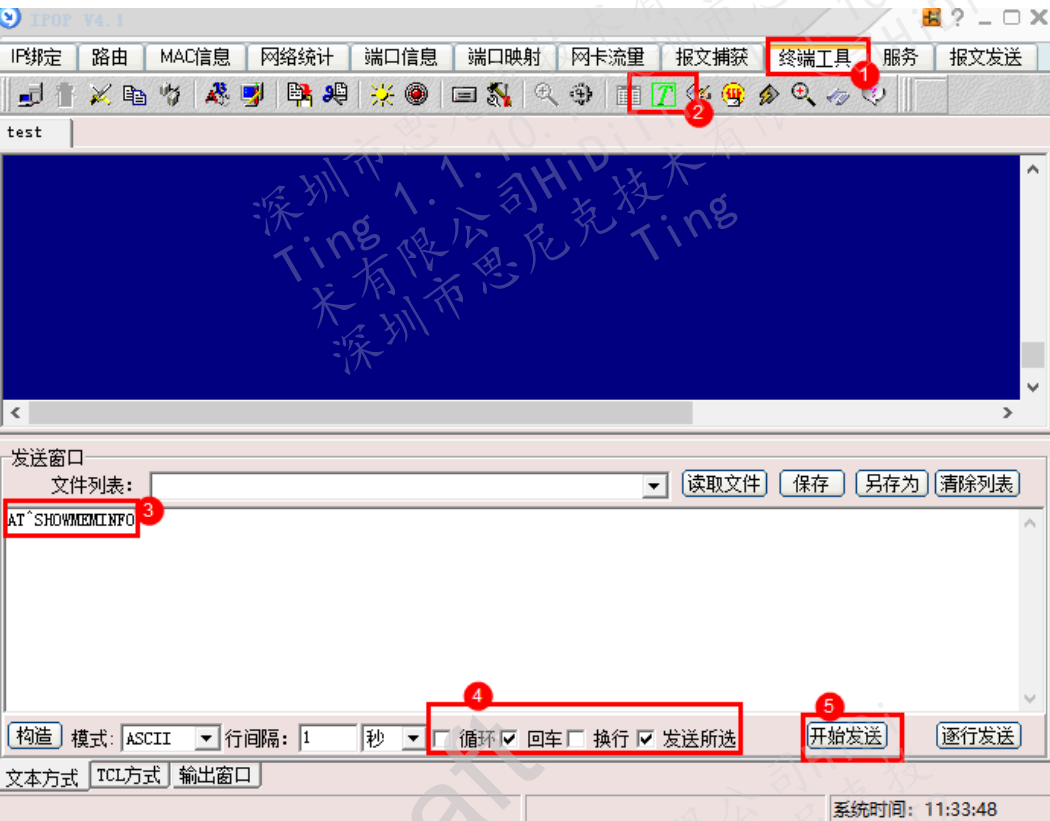
```
[SysHeap stat] total:0x6fffd94, used:0x14f6b0, current free:0x5b06e4, peak usage:0x14fd8c, peak free:0x5b0008
Idx TaskName current malloc peak malloc
00 Swt_Task 00000000 00000024
01 IdleCore000 00000000 00000000
02 system_wq 00000000 00000000
03 sap_msg 00000000 00000000
04 nv_write_task 00000000 00000000
05 BROADCAST 00000000 00000000
06 app 00001252 00001252
07 log 00000000 00015744
08 bt_sdk 00000000 00000000
09 at_cmd 00000000 00000000
10 at_graphic 00000000 00000000
11 at_app 00000000 00000000
12 at_bt_manager 00000000 00000000
13 dfx_msg 00000000 00000000
14 transmit 00000000 00000000
15 power_display 00000000 00000000
16 save_file 00000000 00000000
17 board_app 00000000 00000000
18 RtcEventThread 00004128 00004128
19 GuiMainThread 00047260 00049324
20 InterruptChkThread 00000000 00000000
21 DispatcherThread 00000000 00000000
22 Bootstrap 00024312 00024364
23 BRDCST 00000000 00000000
24 rtcSERVICE 00000000 00000000
25 uiservice 00000000 00000000
26 MsgCenter 00000000 00000000
27 abilityms 00836672 00838564
28 bundles 00000284 00002488
29 BT_SERVICE_TASK 00000000 00000080
30 spp_server_accept_task 00000000 00000000
-----non-task alloc(e.g. startup stage, interrupt)-----
40 00007716 00007716
```

----结束

4.2.5 查询内存的所有信息

步骤 1 使用任意串口工具下发“AT^SHOWMEMINFO”命令，以 IPOP 工具为例，如图 4-9 所示。

图4-9 串口命令



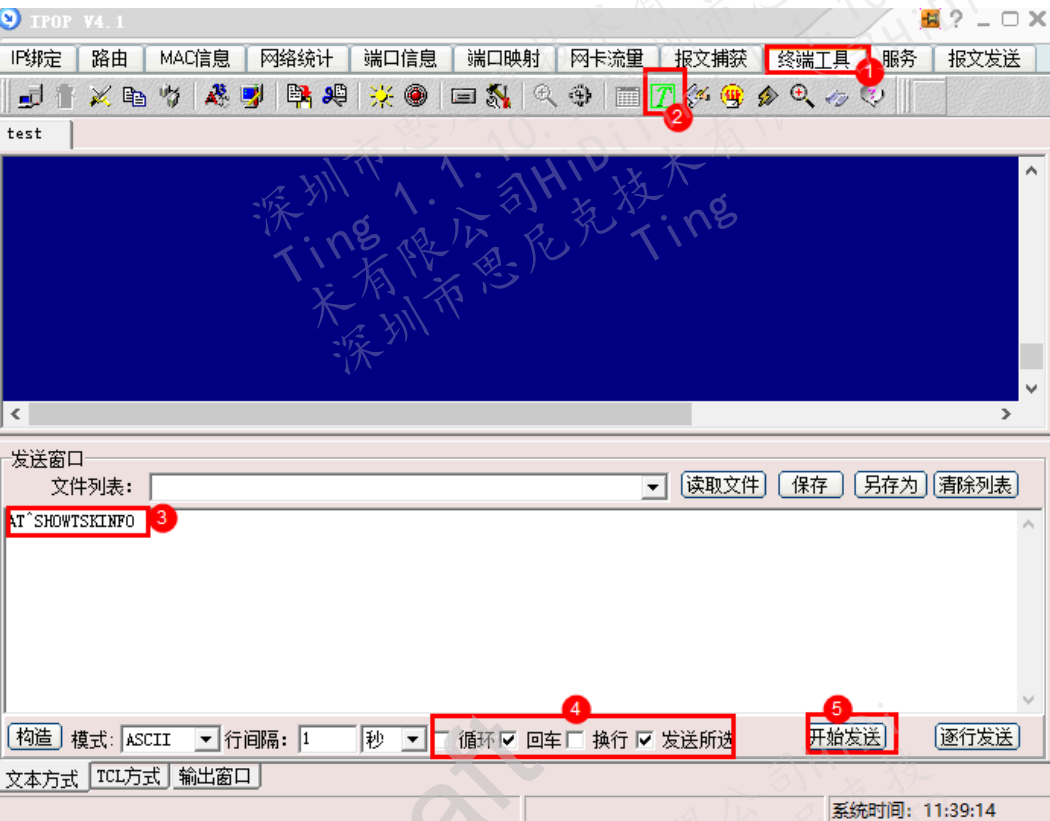
步骤 2 该命令执行结果包含 AT^SHOWTSKWTLS、AT^SHOWHEAPST、AT^SHOWALLTSKHEAP、AT^SHOWMEMFREE 四个命令的执行结果，详情请参考各命令执行结果。

----结束

4.2.6 查询任务信息

步骤 1 使用任意串口工具下发 “AT^SHOWMEMINFO” 命令，以 IPOP 工具为例，如图 4-10 所示。

图4-10 串口命令



步骤 2 命令执行结果包括所有任务的任务名及 taskId，如图 4-11 所示。

图4-11 命令执行结果

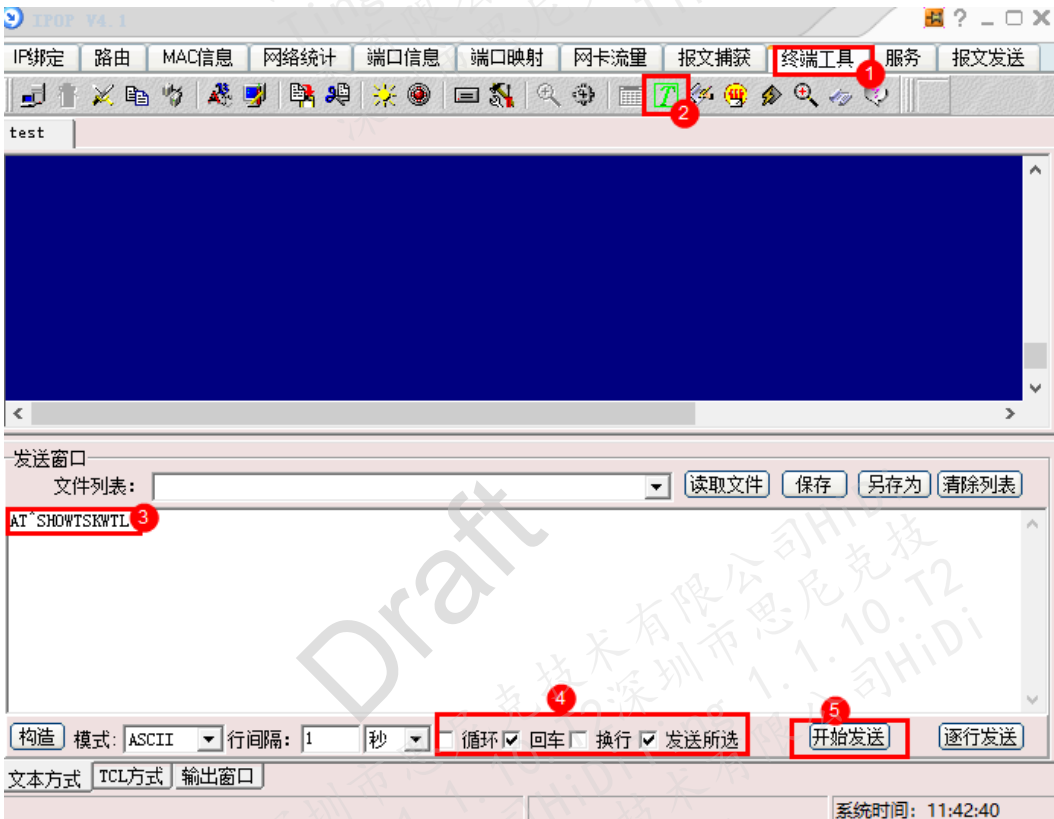
```
task_id: 0, task_name: Swt_Task
task_id: 1, task_name: IdleCore000
task_id: 2, task_name: system_wq
task_id: 3, task_name: sap_msg
task_id: 4, task_name: nv_write_task
task_id: 5, task_name: BROADCAST
task_id: 6, task_name: app
task_id: 7, task_name: log
task_id: 8, task_name: bt_sdk
task_id: 9, task_name: at_cmd
task_id: 10, task_name: at_graphic
task_id: 11, task_name: at_app
task_id: 12, task_name: at_bt_manager
task_id: 13, task_name: dfx_msg
task_id: 14, task_name: transmit
task_id: 15, task_name: power_display
task_id: 16, task_name: save_file
task_id: 17, task_name: board_app
task_id: 18, task_name: RtcEventThread
task_id: 19, task_name: GuiMainThread
task_id: 20, task_name: InterruptChkThread
task_id: 21, task_name: DispatcherThread
task_id: 22, task_name: Bootstrap
task_id: 23, task_name: BRDCST
task_id: 24, task_name: rtcservice
task_id: 25, task_name: uiservice
task_id: 26, task_name: MsgCenter
task_id: 27, task_name: abilityms
task_id: 28, task_name: bundlems
task_id: 29, task_name: BT_SERVICE_TASK
task_id: 30, task_name: spp_server_accept_task
```

----结束

4.2.7 查询任务水线

步骤 1 使用任意串口工具下发“AT^SHOWTSKWTL”命令，以 IPOP 工具为例，如图 4-12 所示。

图4-12 串口命令



步骤 2 命令执行结果包含所有任务的任务栈分配情况如图 4-13 所示。

参数说明如下：

- stackTop: 任务栈初始化时的栈顶;
- stackLen: 任务栈大小;
- peakUsage: 峰值任务栈使用量;
- sp: 当前任务栈栈顶;
- peakRatio: 任务栈峰值使用率。

图4-13 命令执行结果

task_id	taskName	stackTop	stackLen	peakUsage	sp	peakRatio
00	Swt_Task	0x2000C000	0x00000C00	0x0000016C	0x2000CAC0	11%
01	IdleCore000	0x2000CC00	0x00000C00	0x000002C4	0x2000D580	23%
02	system_wq	0x20026A40	0x00000800	0x000001A4	0x20027100	20%
03	sap_msg	0x2002E790	0x00000B00	0x0000064C	0x2002F120	57%
04	nv_write_task	0x20027580	0x00001000	0x00000184	0x20028400	09%
05	BROADCAST	0x6C93A5D0	0x00000800	0x00000444	0x6C93ABE0	53%
06	app	0x20037C60	0x00000C00	0x0000088C	0x200386E0	71%
07	log	0x20038880	0x00000C00	0x000003C4	0x20039300	31%
08	bt_sdk	0x200394A0	0x00000C00	0x000002F4	0x20039F10	24%
09	at_cmd	0x2003C220	0x00000C00	0x000001BC	0x2003CC70	14%
10	at_graphic	0x6C932420	0x00000C00	0x00000184	0x6C932EA0	12%
11	at_app	0x6C933040	0x00001000	0x000006AC	0x6C933F30	41%
12	at_bt_manager	0x2003CE40	0x00001000	0x00000184	0x2003DCC0	09%
13	dfx_msg	0x2003DE60	0x00002000	0x00000374	0x2003FCD0	10%
14	transmit	0x2003FE80	0x00002000	0x00000194	0x20041CF0	04%
15	power_display	0x20041EA0	0x00000C00	0x00000184	0x20042920	12%
16	save_file	0x20042AC0	0x00000C00	0x00000164	0x20043560	11%
17	board_app	0x20043A00	0x00000800	0x000001BC	0x20044060	21%
18	RtcEventThread	0x20044270	0x00000800	0x000002A4	0x200448D0	33%
19	GuiMainThread	0x20044A90	0x00006000	0x00003EB8	0x2004A8E0	65%
20	InterruptChkThread	0x2004B450	0x00000A00	0x000001C4	0x2004BC90	17%
21	DispatcherThread	0x2004BE70	0x00000C00	0x000001E4	0x2004C890	15%
22	Bootstrap	0x6C930D30	0x00001200	0x00000B64	0x6C931D40	63%
23	BRDCST	0x6C939DB0	0x00000800	0x000002B4	0x6C93A3C0	33%
24	rtcservice	0x6C93ADF0	0x00000C00	0x000001F4	0x6C93B800	16%
25	uiservice	0x6C93BA10	0x00000C00	0x000001F4	0x6C93C420	16%
26	MsgCenter	0x6C93C630	0x00000E00	0x000001F4	0x6C93D240	13%
27	abilityms	0x6C93D450	0x00001000	0x00000CC4	0x6C93E260	79%
28	bundlems	0x6C93E470	0x00001C00	0x0000096C	0x6C93FE80	33%
29	BT_SERVICE_TASK	0x2004E150	0x00001000	0x00000444	0x2004EFC0	26%
30	spp_server_accept_task	0x2004F9B0	0x00000800	0x0000038C	0x20050020	44%

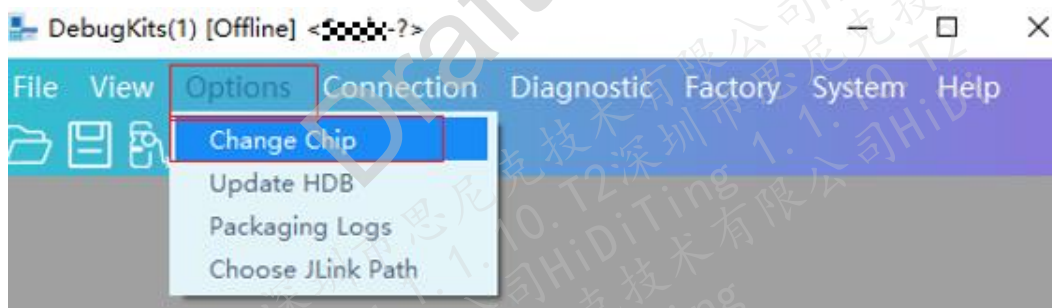
----结束

A DebugKits 使用说明

DIAG 大部分维测功能都需要配合 DebugKits 工具使用，简要介绍一下 DebugKits 工具的使用方法。（详情请参见《HiDiTingV100 DebugKits 工具 使用指南》）

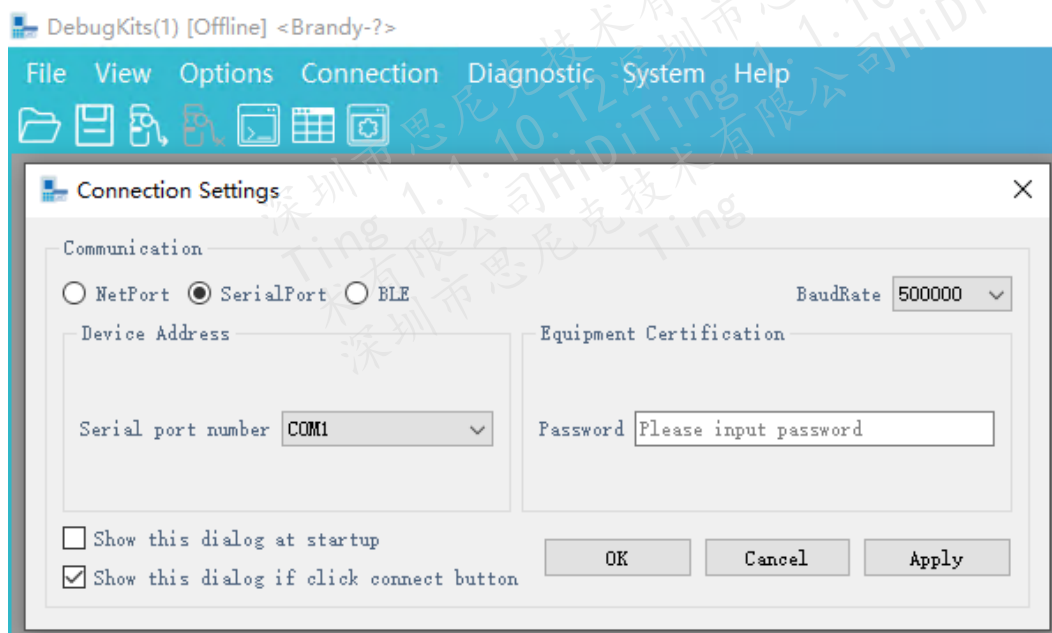
步骤 1 进入工具页面，单击“Options”再选择“Change Chip”，在弹框中选择芯片类型，如图 A-1 所示。

图A-1 选择芯片



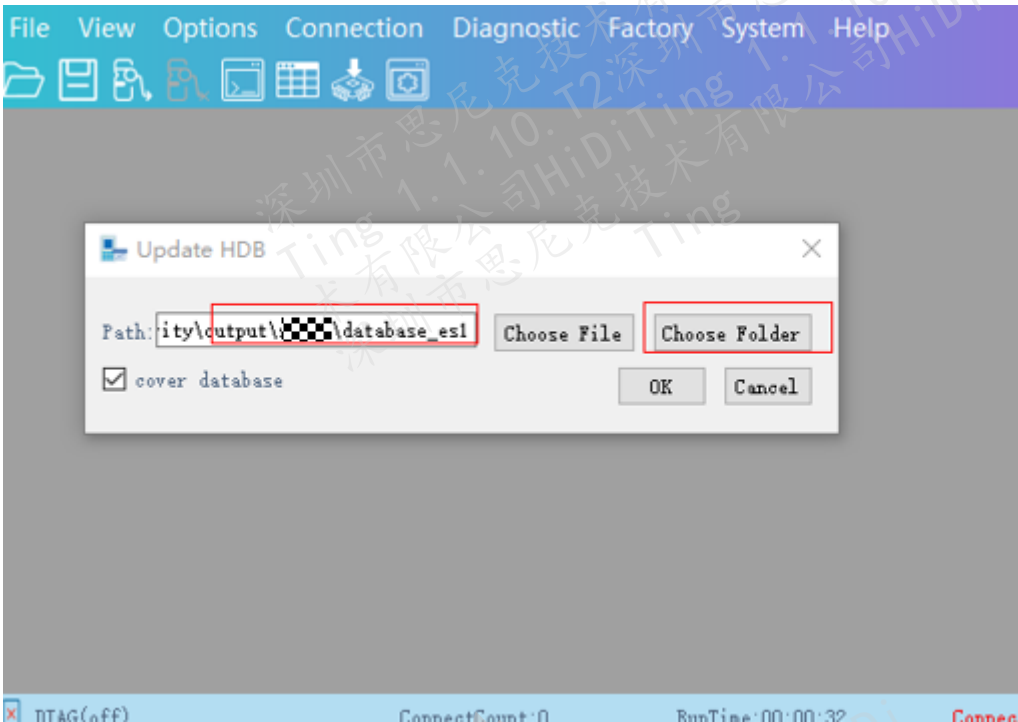
步骤 2 单击“Connection”选择“Connect”在弹窗中选择连接方式以及对应的通道序号进行连接，如图 A-2 所示。

图A-2 连接芯片



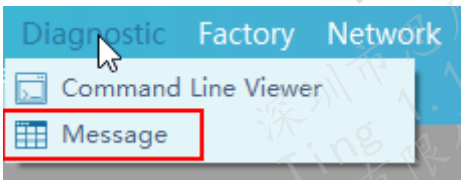
步骤 3 更新数据库。如果是第一次连接芯片或者芯片有新版本的程序，需要更新 HDB 并重新连接之后，日志的打印才能正确的显示，在 Option 菜单下选择“Update HDB”，选择生成的 database 目录（output\brandy\database_evb）再单击“OK”即可完成配置，具体如图 A-3 所示。

图A-3 配置 database

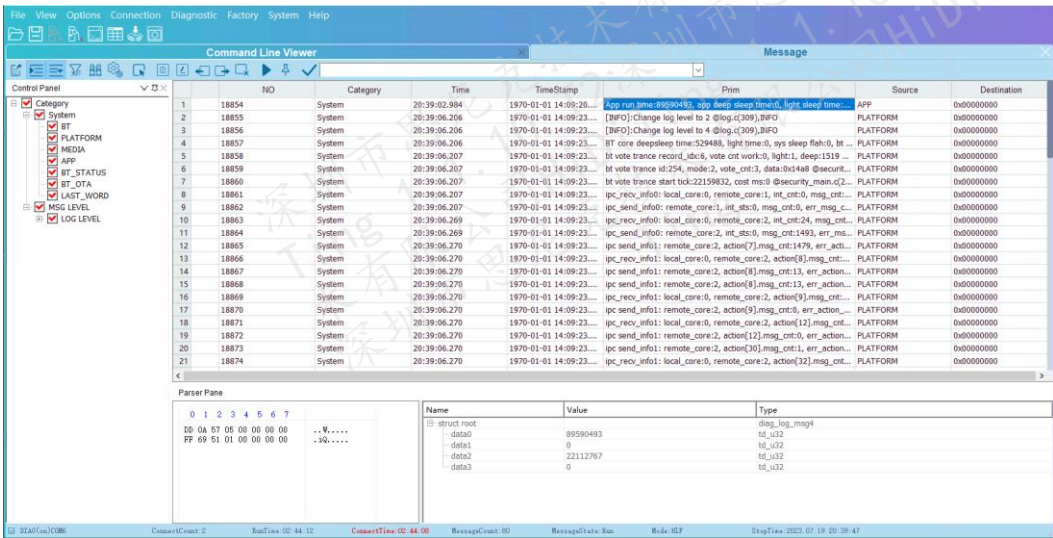


步骤 4 消息界面。单击菜单栏或工具栏上的命令行图标，可打开消息界面查看日志。

图A-4 打开消息界面

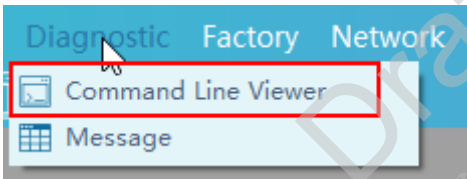


图A-5 查看日志



步骤 5 命令行界面。单击菜单栏或工具栏上的命令行图标，可打开命令行界面。

图A-6 打开命令行界面



图A-7 命令输入



----结束